SAL College of Engineering

CE, IT & ICT Department

Computer Organization Architecture (BE04000251)

Laboratory Manual

Year: 2026

INDEX

Sr. No.	Experiment	Page No.		Date	Marks	Signature
		From	To			
1	Introduction of Digital Logic gate. Write a program in C language for functionality of digital logic gate.					
2	Implement binary division algorithm					
3	Implement Booth's multiplication algorithm.					
4	Study of Simulator GNU Sim8085 and Architecture of 8085.					
5	Write an assembly language code in GNU sim8085 to store number in reverse order in memory location.					
6	Write an assembly language code in GNU sim8085 to implement arithmetic Instruction.					
7	Write an assembly language code in GNU sim8085 to find the factorial of a number.					
8	Write an assembly language code in GNU sim8085 to implement logical Instructions					
9	Design ALU using Logisim.					
10	Implement 16-bit single-cycle MIPS processor in Verilog HDL					

EXPERIMENT NO: 1 DATE: / / 2026

TITLE: Introduction of Digital Logic Gate. Write a program in C Language for functionality of digital logic gates.

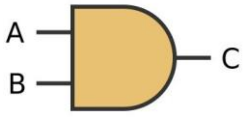
OBJECTIVES: On completion of this experiment student will able to…

- Know the concept of Digital Logic Gate
- Create a program for functionality of logic gates.

THEORY:

- **Introduction of Digital Logic Gates:**
  1. **AND Gate:** AND Gate performs the logical AND operation. It accepts the combination of two or more input variables and performs multiplication of those input variables to provide a single output signal.

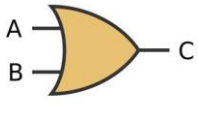
     Block Diagram of AND Gate.



A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

  2. **OR Gate**: OR Gate performs the addition operation between the two or more binary input variables. An OR Gate consists of two or more inputs and produces one output which is the sum of all those binary input variables.

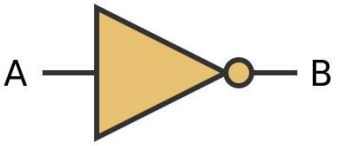
     Block Diagram of OR Gate:



A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

  3. **NOT Gate:** NOT Gate performs the operations like a logical inverter. So, the output will be the complement of the input. NOT Gate has a different behavior as compared to other gates since it contains only one input signal.

Block Diagram of NOT Gate:



A	B
0	1
1	0

EXAMPLE-1:

Write a Menu Driven program in C Language to implement the digital gates. The program should give the truth table of the selected gate.

```
// C program implementing the AND, OR, NOT and EX-OR gates
#include <stdio.h>
#include <stdlib.h>
int main ()
{
int n, i, A[4], B[4], Y[4];
printf ("Select the logic gate you want: \n");
printf ("\n 1- AND gate. \n 2- OR gate. \n 3- NOT gate. \n 4- EX-OR gate. \n");
scanf ("%d", &n);
switch(n)
{
case 1:
printf ("Enter the two binary inputs (0 or 1) for AND gate: \n");
for (i = 0; i < 4; i++)
    {
        printf ("Input A: \n");
        scanf ("%d", &A[i]);
        printf ("Input B: \n");
        scanf ("%d", &B[i]);
        printf ("Output Y:\n");
        Y[i] = A[i] & B[i];
        printf ("%d \n", Y[i]);
    }
printf ("The truth table of AND Gate is as below:\n");
printf ("\nA\tB\tY= (A.B)");
printf ("\n%d\t%d\t%d", A[0], B[0], Y[0]);
printf ("\n%d\t%d\t%d", A[1], B[1], Y[1]);
printf ("\n%d\t%d\t%d", A[2], B[2], Y[2]);
printf ("\n%d\t%d\t%d", A[3], B[3], Y[3]);
break;
case 2:
```

```
printf ("Enter the two binary inputs (0 or 1) for OR gate: \n");
for (i = 0; i < 4; i++)
     {
          printf ("Input A: \n");
          scanf ("%d", &A[i]);
          printf ("Input B: \n");
          scanf ("%d", &B[i]);
          printf ("Output Y:\n");
          Y[i] = A[i] | B[i];
          printf ("%d \n", Y[i]);
     }
printf ("The truth table of OR Gate is as below:\n");
printf ("\nA\tB\tY= (A + B)");
printf ("\n%d\t%d\t%d", A[0], B[0], Y[0]);
printf ("\n%d\t%d\t%d", A[1], B[1], Y[1]);
printf ("\n%d\t%d\t%d", A[2], B[2], Y[2]);
printf ("\n%d\t%d\t%d", A[3], B[3], Y[3]);
break;
case 3:
printf ("Enter the one binary input (0 or 1) for NOT gate: \n");
for (i = 0; i < 2; i++)
     {
          printf ("Input A: \n");
          scanf ("%d", &A[i]);
          printf ("Output Y:\n");
          Y[i]= ! (A[i]);
          printf ("%d \n", Y[i]);
     }
printf ("The truth table of NOT Gate is as below:\n");
printf ("\nA\tY= !(A)'");
printf ("\n%d\t%d", A[0], Y[0]);
printf ("\n%d\t%d", A[1], Y[1]);
break;
case 4:
printf ("Enter the two binary inputs (0 or 1) for EX-OR gate: \n");
for (i = 0; i < 4; i++)
     {
          printf ("Input A: \n");
          scanf ("%d", &A[i]);
          printf ("Input B: \n");
          scanf ("%d", &B[i]);
          printf ("Output Y:\n");
          Y[i] = A[i] ^ B[i];
```

```
            printf ("%d \n", Y[i]);
        }
printf ("The truth table of EX-OR Gate is as below:\n");
printf ("\nA\tB\tY= (A ^ B)");
printf ("\n%d\t%d\t%d", A[0], B[0], Y[0]);
printf ("\n%d\t%d\t%d", A[1], B[1], Y[1]);
printf ("\n%d\t%d\t%d", A[2], B[2], Y[2]);
printf ("\n%d\t%d\t%d", A[3], B[3], Y[3]);
break;
default: printf ("Invalid!!");
}
return 0;
}
```

Output:

```
Select the logic gate you want:
 1- AND gate.
 2- OR gate.
 3- NOT gate.
 4- EX-OR gate.
2
Enter the two binary inputs (0 or 1) for OR gate:
Input A:
0
Input B:
0
Output Y:
0
Input A:
0
Input B:
1
Output Y:
1
Input A:
1
Input B:
0
Output Y:
1
Input A:
1
Input B:
1
Output Y:
1
```

```
The truth table of OR Gate is as below:
A   B   Y= (A + B)
0   0   0
0   1   1
1   0   1
1   1   1
=== Code Execution Successful ===
```

EXCERCISE:

1) Write a program in C Language to convert decimal into binary number.
2) Write a program in C Language to convert binary into decimal number.

QUIZ:

1) A digital computer has a common bus system for 16 registers of 32 bits each. The bus is constructed with multiplexers.
   - **A.** How many selection inputs are there in each multiplexer?
   - **B.** What size of multiplexers are needed?
   - **C.** How many multiplexers are there in the bus?
2) Identify the memory operation in each case and explain it.
   - **A.** R2 ← M[AR]
   - **B.** M[AR] ← R3
   - **C.** R5 ← M[R5]

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____

EXPERIMENT NO: 2 DATE: / / 2026

TITLE: To implement division algorithm.

OBJECTIVES: On completion of this experiment student will able to…

- Learn division algorithm.
- Create restoring division algorithm for unsigned integer.

THEORY:

Restoring Division Algorithm For Unsigned Integer

- **Step-1**: First the registers are initialized with corresponding values (Q = Dividend, M= Divisor, A = 0, n = number of bits in dividend)
- **Step-2:** Check the sign bit of register A.
- **Step-3:** If it is 1 shift left content of AQ and perform A = A+M, otherwise shift left AQ and perform A = A-M (means add 2's complement of M to A and store it to A).
- **Step-4:** Again, check the sign bit of register A.
- **Step-5:** If sign bit is 1 Q [0] become 0 otherwise Q [0] become 1 (Q [0] means least significant bit of register Q).
- **Step-6:** Decrements value of N by 1.
- **Step-7:** If N is not equal to zero go to Step 2 otherwise go to next step.
- **Step-8:** If sign bit of A is 1 then perform A = A+M.
- **Step-9:** Register Q contain quotient and A contain remainder.

Example: Dividend (A) = 1011, i.e. 11, and Divisor (B) = 00011, i.e. 03.

Action	M	A	Q	Count
Initial	00011	00000	1011	4
Shift Left AQ	00011	00001	011_	
A = A - M	00011	11110 (A= 00001 M= 00011 Take 2's complement of M 11101 so add in A 00001	011_	

		+11101 11110		
Q [0] = 0 And restore A, Decrement counter by 1	00011	00001	0110	3
shift left AQ	00011	00010	110_	
A=A-M	00011	11111	110_	
Q [0] = 0 and restore A, Decrement counter by 1	00011	00010	1100	
shift left AQ	00011	00101	100_	2
A=A-M	00011	00010	100_	
Q [0] = 1and Decrement counter by 1	00011	00010	1001	
shift left AQ	00011	00101	001_	1
A=A-M	00011	00010	001_	
Q [0] = 1	00011	00010	0011	

A	Q	M = 0011	
0000	0111	Initial values	
0000	1110	Shift	1
1101		Subtract	
0000	1110	Restore	
0001	1100	Shift	2
1110		Subtract	
0001	1100	Restore	
0011	1000	Shift	3
0000		Subtract	
0000	1001	$Q_0 = 1$	
0001	0010	Shift	4
1110		Subtract	
0001	0010	Restore	

(7) / (3)

A	Q	M = 1101	
0000	0111	Initial values	
0000	1110	Shift	1
1101		Add	
0000	1110	Restore	
0001	1100	Shift	2
1110		Add	
0001	1100	Restore	
0011	1000	Shift	3
0000		Add	
0000	1001	$Q_0 = 1$	
0001	0010	Shift	4
1110		Add	
0001	0010	Restore	

(7) / (-3)

Example-1:

Write a program in C language to implement division algorithm (Using of Restoring method).

```
#include <stdio.h>
void binout_helper (int in, int left)
{
    if (left == 0) return;
    binout_helper (in >> 1, left - 1);
    printf ("%d", in & 1);
}
void binout (int in)
{
    binout_helper(in,8);
}
void print_status (int Q, int A, int M)
{
    printf ("Q: %3d, A: %3d, M: %3d [", Q, A, M);
    binout(A);
    printf (" ");
    binout(Q);
    printf ("] \n");
}
int main ()
{
    unsigned int i, dividend, divisor, A, Q, M, AQ;
    printf ("Enter Dividend: ");
    scanf ("%d", &dividend);
    printf ("Enter Divisor: ");
    scanf ("%d", &divisor);
    A = 0;
    Q = dividend;
    M = divisor;
    print_status (Q, A, M);
    for (i = 0; i < 8; i++)
    {
        // Step 1: Shift A and Q left
        AQ = (A << 8) + Q;
        AQ <<= 1;
        A = (AQ >> 8) & 0xFF;
        Q = AQ & 0xFF;
        // Step 2: Subtract M from A, place answer in A
        A -= M;
        if (A & (1 << 7))
        {
            // If the sign bit of A is 1, set q_0 to 0
            Q &= ~1;
```

```
            // and add M back to A (restore A)
            A += M;
        }
        else
        {
            // otherwise, set q_0 to 1
            Q |= 1;
        }
        print_status (Q, A, M);
    }
    printf("Quotient(Q): ");
    binout(Q);
    printf("\t(%d)", Q);
    printf ("\n Remainder(A): ");
    binout(A);
    printf("\t(%d)", A);
    return 0;
}
```

Output:

```
Enter Dividend: 128
Enter Divisor: 6
Q: 128, A:   0, M:   6 [00000000 10000000]
Q:   0, A:   1, M:   6 [00000001 00000000]
Q:   0, A:   2, M:   6 [00000010 00000000]
Q:   0, A:   4, M:   6 [00000100 00000000]
Q:   1, A:   2, M:   6 [00000010 00000001]
Q:   2, A:   4, M:   6 [00000100 00000010]
Q:   5, A:   2, M:   6 [00000010 00000101]
Q:  10, A:   4, M:   6 [00000100 00001010]
Q:  21, A:   2, M:   6 [00000010 00010101]
Quotient(Q): 00010101   (21)
Remainder(A): 00000010      (2)
=== Code Execution Successful ===
```

EXERCISES:

1) Write a program in C language to implement division algorithm (Using of non-restoring method).

QUIZ:

1) Using of adder-sub tractor circuit. Following values for input mode M and data inputs A and B. In each case, determine the values of the outputs: S3, S2, S1, S0 and C4.

M	A	B	M	A	B	M	A	B
a. 0	0111	0110	b. 0	1000	1001	c.1	1100	1000
d. 1	0101	1010	e. 1	0000	0001			

2) Dividend (A) = 1100, i.e. 12, and Divisor (B) = 00011 Give the each and every step of restore division algorithm and display answer.

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____

EXPERIMENT NO: 3 DATE: / / 2026

TITLE: Implement Booth's Algorithm

OBJECTIVES: On completion of this experiment student will able to…

- Understanding of multiplication algorithm.
- Multiplication with use of 2's complement.

THEORY:

Booth algorithm gives a procedure for multiplying binary integers in signed-2's complement representation. It operates on the fact that strings of 0's in the multiplier require no addition but just shifting, and a string of 1's in the multiplier from bit weight 2^k to weight 2^m can be treated as $2^{k+1}-2$

The Below table is for 7*3

A	Q	Q_{-1}	M		
0000	0011	0	0111	Initial values	
1001	0011	0	0111	A ← A − M	First cycle
1100	1001	1	0111	Shift	
1110	0100	1	0111	Shift	Second cycle
0101	0100	1	0111	A ← A + M	Third cycle
0010	1010	0	0111	Shift	
0001	0101	0	0111	Shift	Fourth cycle

0111			0111		
×0011	(0)		×1101	(0)	
11111001	1–0		11111001	1–0	
0000000	1–1		0000111	0–1	
000111	0–1		111001	1–0	
00010101	(21)		11101011	(−21)	

(a) (7) × (3) = (21) (b) (7) × (−3) = (−21)

1001			1001		
×0011	(0)		×1101	(0)	
00000111	1–0		00000111	1–0	
0000000	1–1		1111001	0–1	
111001	0–1		000111	1–0	
11101011	(−21)		00010101	(21)	

(c) (−7) × (3) = (−21) (d) (−7) × (−3) = (21)

Example-1:

Write a program in C language to implement multiplication algorithm with use of booth's algorithm.

```
#include <stdio.h>
#include <math.h>
 int a = 0,b = 0, c = 0, a1 = 0, b1 = 0, com[5] = { 1, 0, 0, 0, 0};
int anum[5] = {0}, anumcp[5] = {0}, bnum[5] = {0};
int acomp[5] = {0}, bcomp[5] = {0}, pro[5] = {0}, res[5] = {0};

void binary()
{
   a1 = fabs(a);
   b1 = fabs(b);
   int r, r2, i, temp;
   for (i = 0; i < 5; i++)
   {
      r = a1 % 2;
      a1 = a1 / 2;
      r2 = b1 % 2;
      b1 = b1 / 2;
     anum[i] = r;
     anumcp[i] = r;
     bnum[i] = r2;
     if(r2 == 0)
      {
         bcomp[i] = 1;
      }
      if(r == 0)
      {
         acomp[i] =1;
      }
   }
   c = 0;
   for ( i = 0; i < 5; i++)
   {
      res[i] = com[i]+ bcomp[i] + c;
      if(res[i] >= 2)
      {
         c = 1;
      }
```

```
    else
    {
    c = 0;
    res[i] = res[i] % 2;
    }
 }
 for (i = 4; i >= 0; i--)
 {
    bcomp[i] = res[i];
 }
 if (a < 0)
 {
    c = 0;
    for (i = 4; i >= 0; i--)
    {
       res[i] = 0;
    }
    for ( i = 0; i < 5; i++)
    {
       res[i] = com[i] + acomp[i] + c;
       if (res[i] >= 2)
       {
          c = 1;
       }
       else
       {
          c = 0;
          res[i] = res[i]%2;
       }
    }
    for (i = 4; i >= 0; i--)
    {
       anum[i] = res[i];
       anumcp[i] = res[i];
    }

 }
if(b < 0)
{
    for (i = 0; i < 5; i++)
    {
       temp = bnum[i];
       bnum[i] = bcomp[i];
```

```
            bcomp[i] = temp;
        }
    }
}
void add(int num[])
{
    int i;
    c = 0;
    for ( i = 0; i < 5; i++)
    {
        res[i] = pro[i] + num[i] + c;
        if (res[i] >= 2)
        {
            c = 1;
        }
        else
        {
            c = 0;
        }
        res[i] = res[i]%2;
    }
    for (i = 4; i >= 0; i--)
    {
        pro[i] = res[i];
        printf("%d",pro[i]);
    }
    printf(":");
    for (i = 4; i >= 0; i--)
    {
        printf("%d", anumcp[i]);
    }
}
void arshift()
{
    int temp = pro[4], temp2 = pro[0], i;
    for (i = 1; i < 5  ; i++)
    {
        pro[i-1] = pro[i];
    }
    pro[4] = temp;
    for (i = 1; i < 5  ; i++)
    {
        anumcp[i-1] = anumcp[i];
```

```
    }
    anumcp[4] = temp2;
    printf("\nAR-SHIFT: ");
    for (i = 4; i >= 0; i--)
    {
        printf("%d",pro[i]);
    }
    printf(":");
    for(i = 4; i >= 0; i--)
    {
        printf("%d", anumcp[i]);
    }
}

void main()
{
    int i, q = 0;
    printf("\t\tBOOTH'S MULTIPLICATION ALGORITHM");
    printf("\nEnter two numbers to multiply: ");
    printf("\nBoth must be less than 16");
    //simulating for two numbers each below 16
    do
    {
        printf("\nEnter A: ");
        scanf("%d",&a);
        printf("Enter B: ");
        scanf("%d", &b);
    }
    while(a >=16 || b >=16);
    printf("\nExpected product = %d", a * b);
    binary();
    printf("\n\nBinary Equivalents are: ");
    printf("\nA = ");
    for (i = 4; i >= 0; i--)
    {
        printf("%d", anum[i]);
    }
    printf("\nB = ");
    for (i = 4; i >= 0; i--)
    {
        printf("%d", bnum[i]);
    }
    printf("\nB'+ 1 = ");
```

```
    for (i = 4; i >= 0; i--)
    {
        printf("%d", bcomp[i]);
    }
    printf("\n\n");
    for (i = 0;i < 5; i++)
    {
        if (anum[i] == q)
        {
            printf("\n-->");
            arshift();
            q = anum[i];
        }
        else if(anum[i] == 1 && q == 0)
        {
            printf("\n-->");
            printf("\nSUB B: ");
            add(bcomp);
            arshift();
            q = anum[i];
        }
        else
        {
            printf("\n-->");
            printf("\nADD B: ");
            add(bnum);
            arshift();
            q = anum[i];
        }
    }
    printf("\nProduct is = ");
    for (i = 4; i >= 0; i--)
    {
        printf("%d", pro[i]);
    }
    for (i = 4; i >= 0; i--)
    {
        printf("%d", anumcp[i]);
    }
}
```

Output:

```
        BOOTH'S MULTIPLICATION ALGORITHM
Enter two numbers to multiply:
Both must be less than 16
Enter A: -12
Enter B: 13
Expected product = -156

Binary Equivalents are:
A = 10100
B = 01101
B'+ 1 = 10011

-->
AR-SHIFT: 00000:01010
-->
AR-SHIFT: 00000:00101
-->
SUB B: 10011:00101
AR-SHIFT: 11001:10010
-->
ADD B: 00110:10010
AR-SHIFT: 00011:01001
-->
SUB B: 10110:01001
AR-SHIFT: 11011:00100
Product is = 1101100100
```

QUIZ:

1) Register A holds the 8bit binary 11011001. Determine the B operand and the logic microoperation to be performed in order to change the value in A to:

A) 01101101 B) 11111101

2) The 8-bit registers AR, BR, CR and DR initially have the following values:

AR = 11110010 BR = 11111111 CR = 10111001 DR = 1110101011

Determine the 8bit values in each register after the execution of the following sequence of microoperations.

A) AR → AR + BR B) CR → CR Λ DR C) BR → BR + 1 D) AR → AR – CR

An output program resides in memory.

EVALUATION:

Involvement (4)	Understanding / Problem solving (3)	Timely Completion (3)	Total (10)

Signature with date: _____

EXPERIMENT NO: 4 DATE: / / 2026

TITLE: Study of Simulator GNU Sim8085 and Architecture of 8085.

OBJECTIVES: On completion of this experiment student will able to…

- know the concept of GNU sim8085.
- Understand the architecture of 8085.

THEORY:

GNU Sim8085:

GNU Sim8085 is a graphical simulator, assembler and debugger for the Intel 8085 microprocessor in Linux and Windows. It contains a simple editor component with syntax highlighting. Also, it has a keypad to input assembly language instructions with appropriate arguments. The image below shows default appearance of the GNUSim8085.

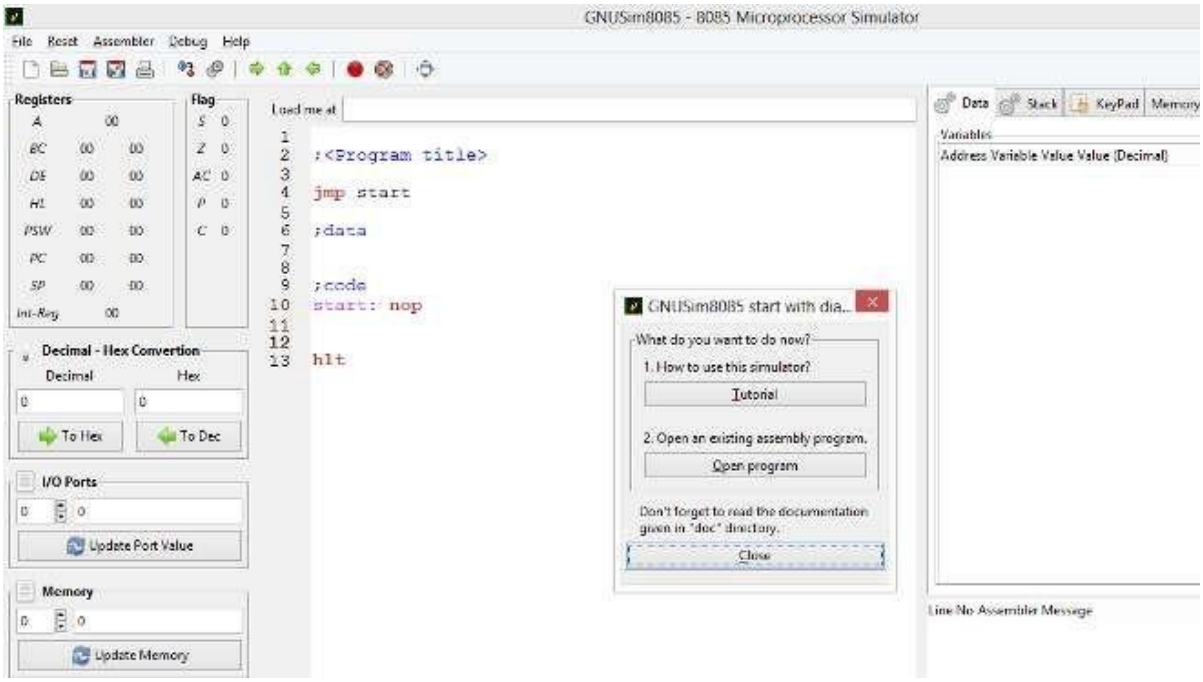


GNUSim8085 is a 8085 microprocessor simulator with following features.

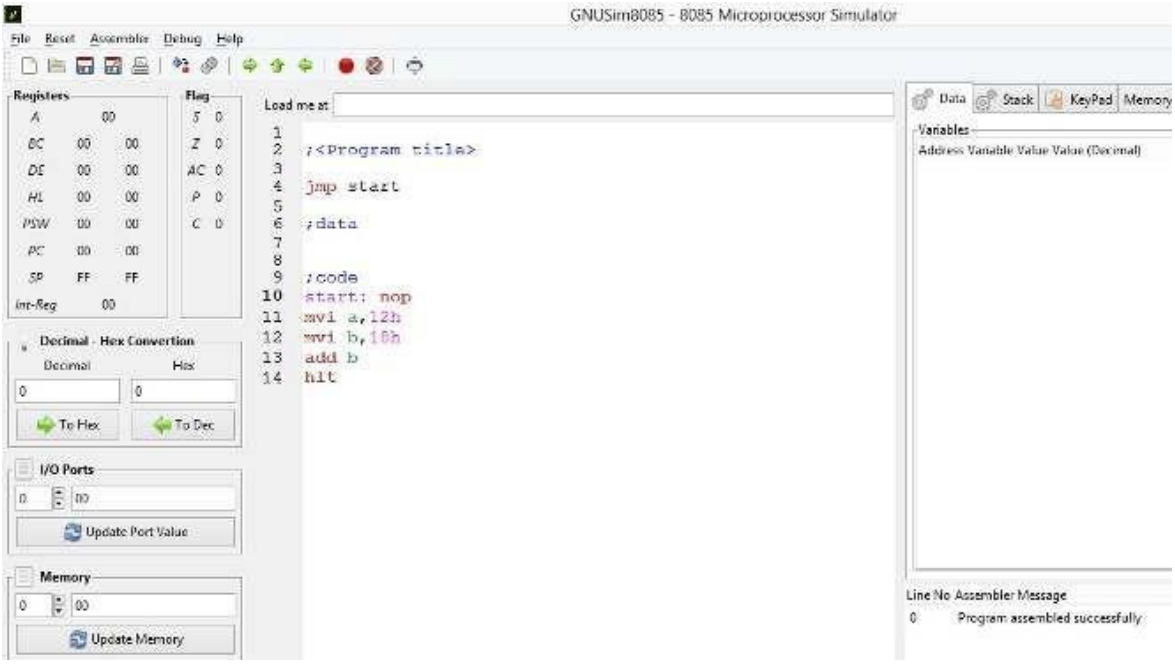
- A simple editor component with syntax highlighting.
- A keypad to input assembly language instructions with appropriate arguments.
- Easy view of register contents.
- Easy view of flag contents.
- Hexadecimal to Decimal and Decimal to Hexadecimal converter.
- View of stack, memory and I/O contents.

- Support for breakpoints for program debugging.
- Stepwise program execution.
- One click conversion of assembly program to opcode listing.
- Printing support.
- UI translated in various languages.

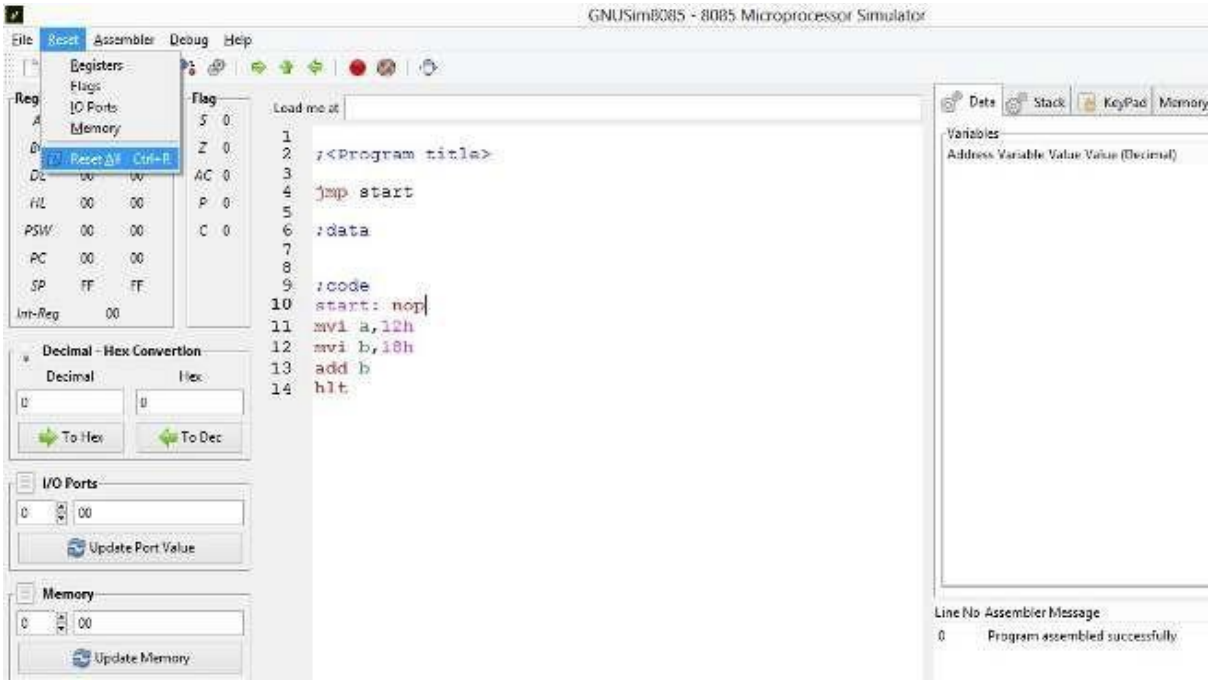
How to start with GNU Simulator 8085:



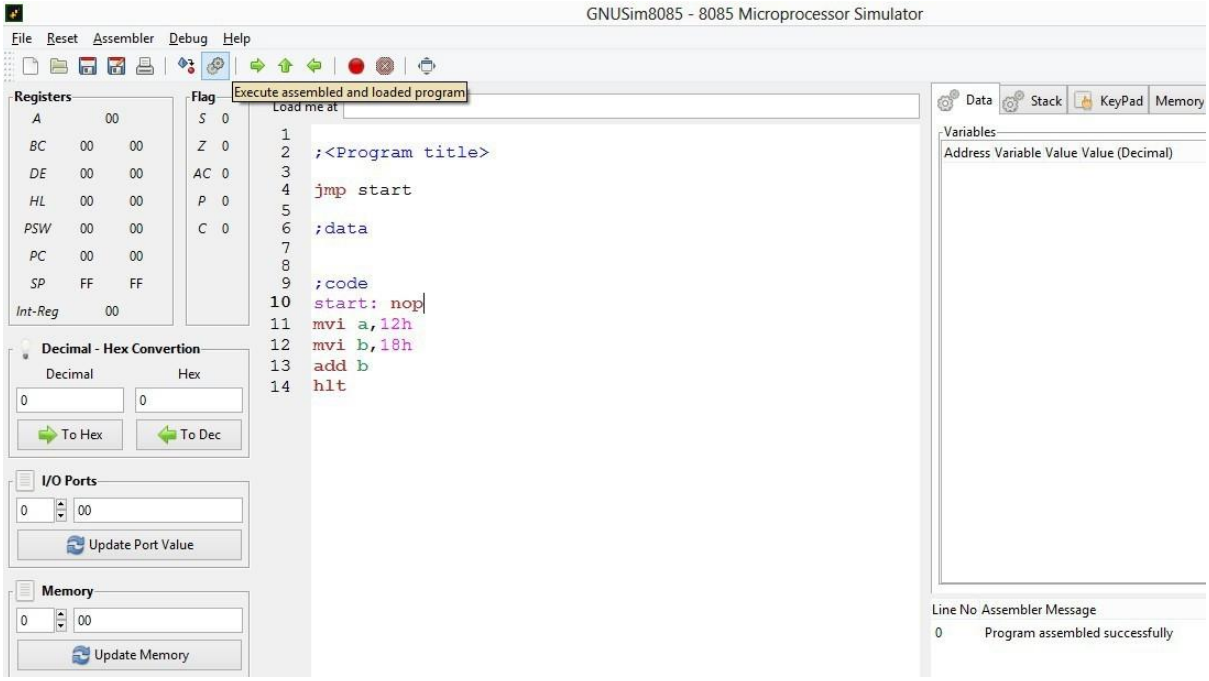
Step1: open GNU Sim 8085 above window will open. Now click on close button highlighted in the above image.



Step2: start writing the code after **start: nop** in load me at 10 that is at load me at 11 in above image.

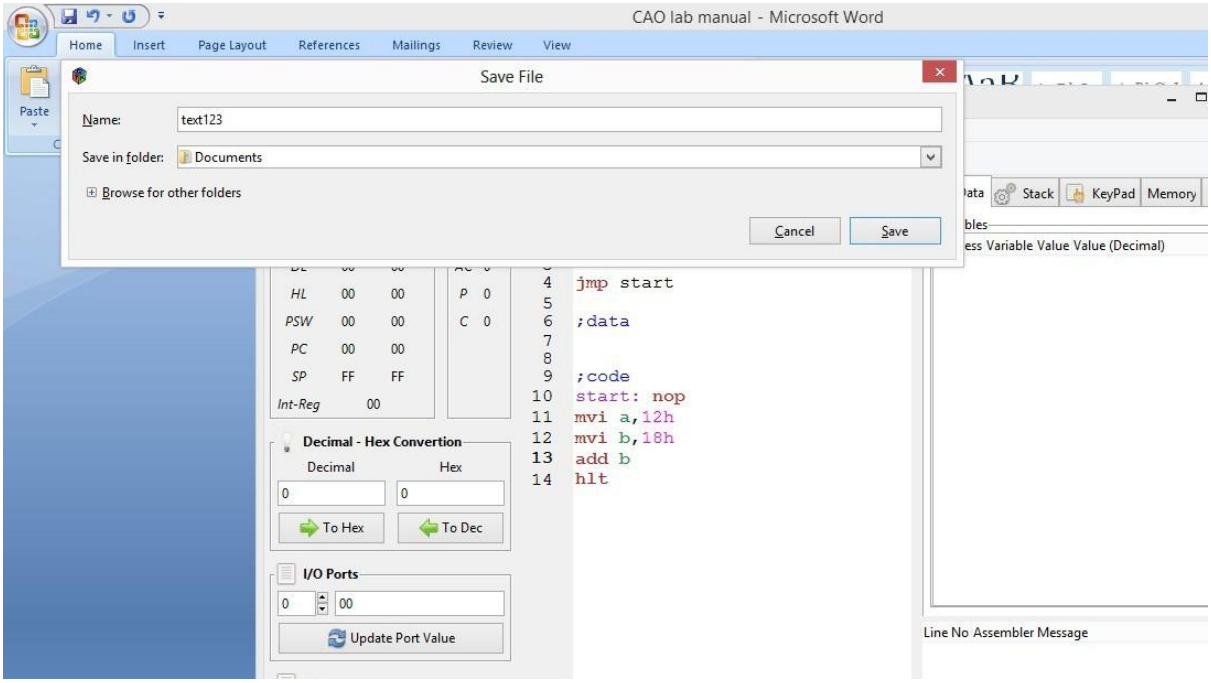


Step 3: click on reset and reset all the registers by clicking on reset all in above image



Step 4: click on the highlighted button to execute the code in above screen.

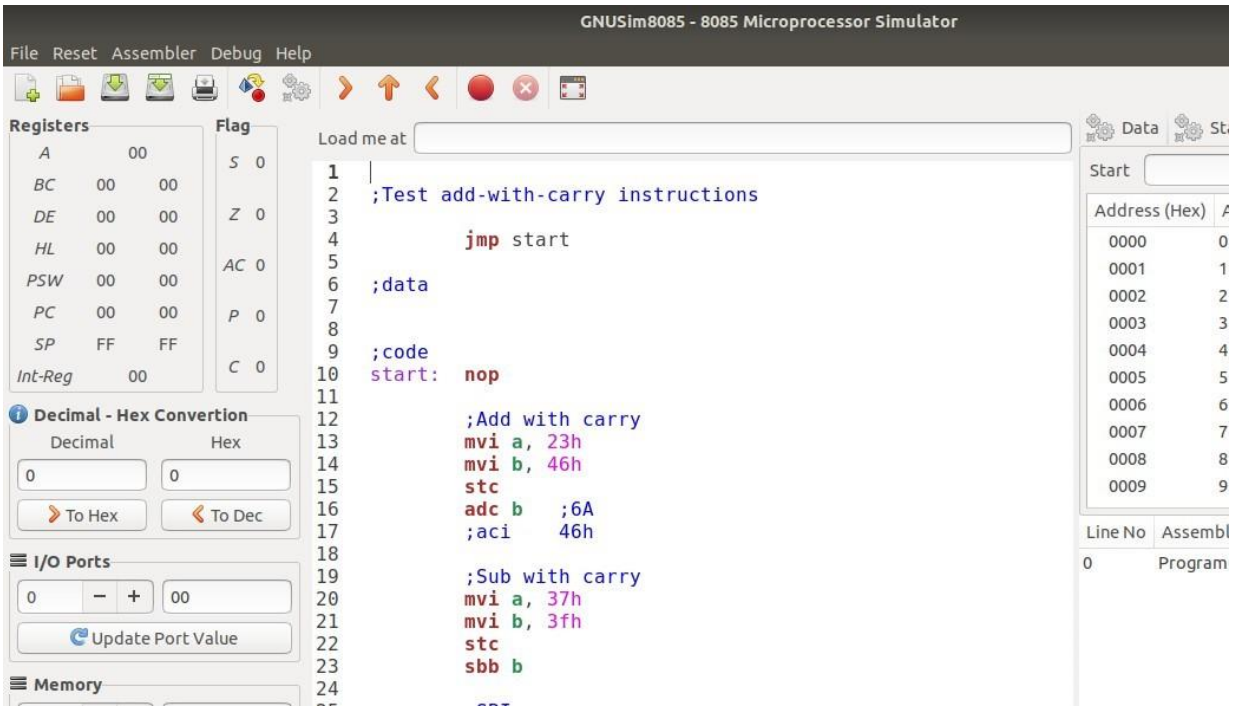
Step 5: after you execute the code mention the name your program by writing the name in the name section as mentioned in the screen shot in picture 5 and the drive where you want to save it. After that click on save in the below image.

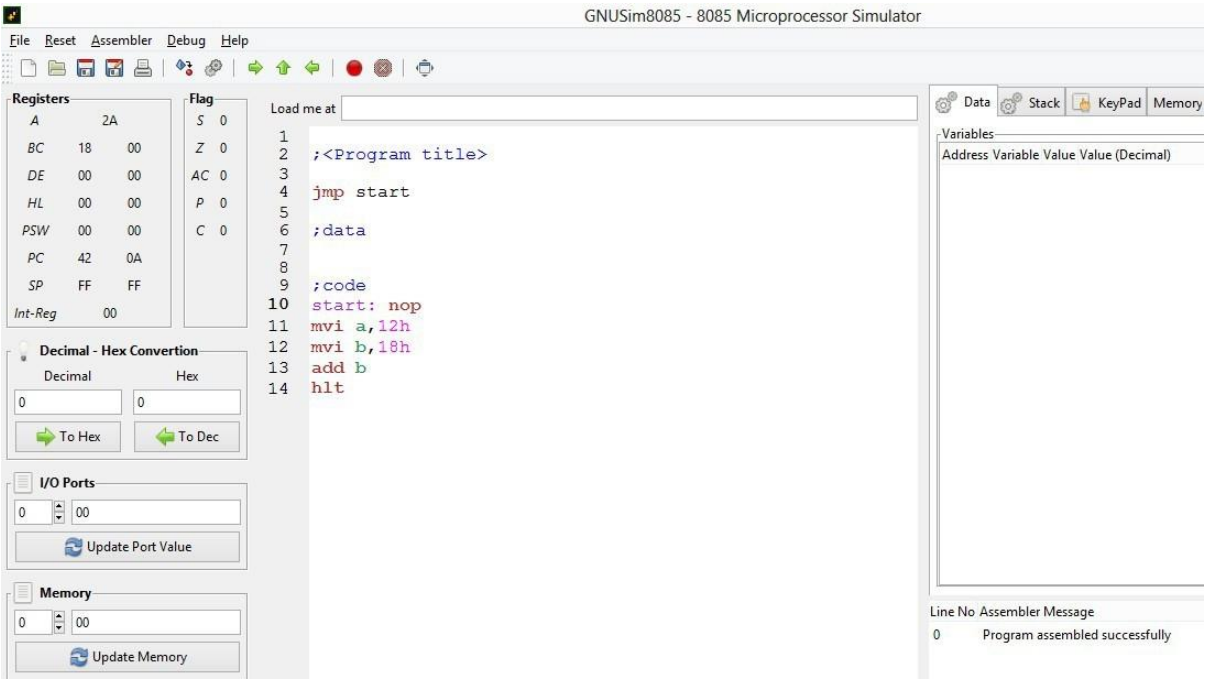


Step 6: after step 5, you will see the result of the instructions in the respective registers as seen in the above image.

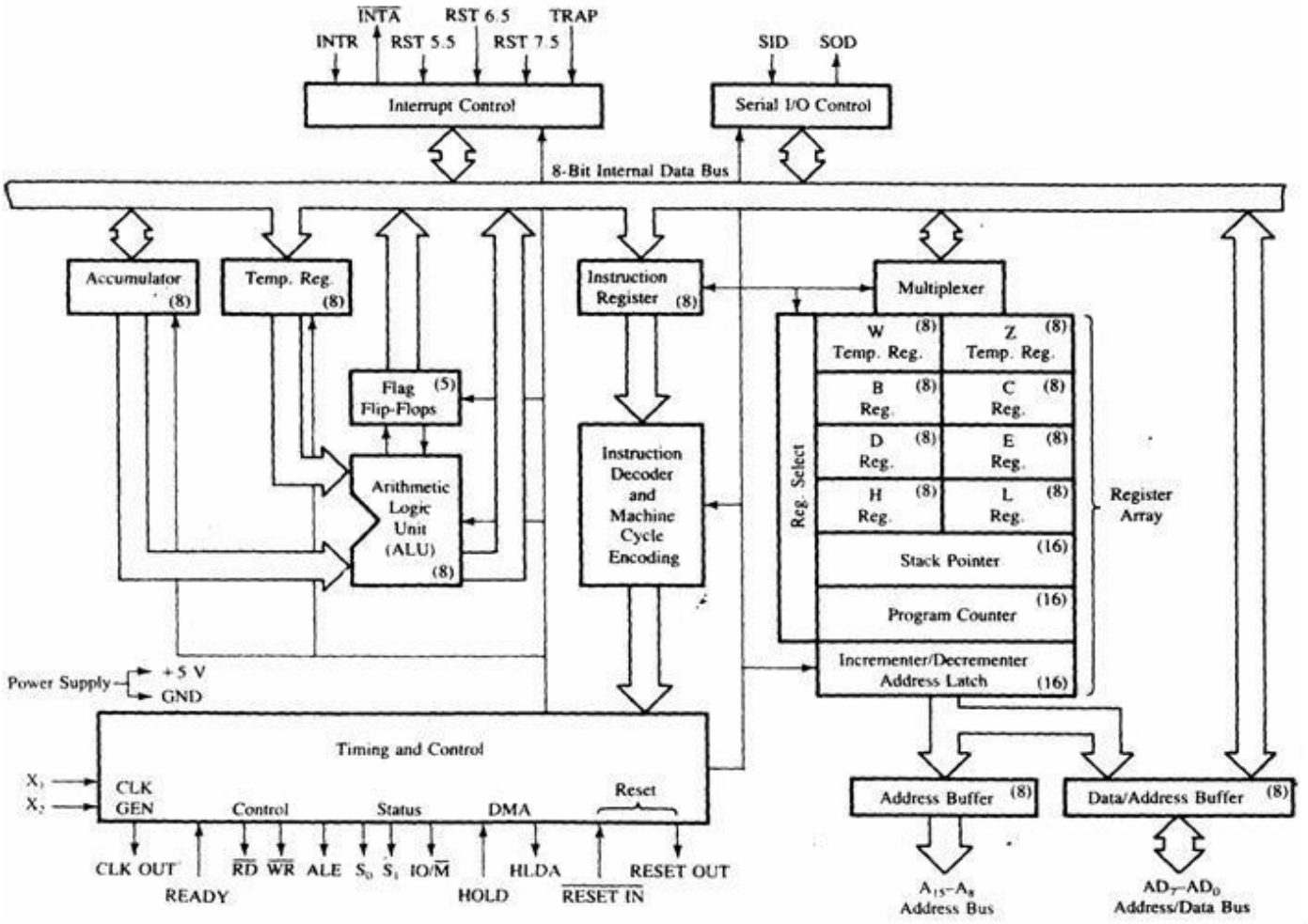
Example:

The image below shows the GNUSim8085 sample addition with carry program; Registers and flags on the left size along with their status values; left below shows decimal to hex conversion with I/O ports and Memory ports with editing accessibility; on the right is loaded address contents.





Basic Architecture 0f 8085 microprocessor:



8085 microprocessor is an 8-bit microprocessor designed by Intel in 1977 using NMOS (N-channel metal-oxide semiconductor) technology.

It has the following features

- 8-bit data bus
- 16-bit address bus, which can address up to 64KB
- A 16-bit program counter
- A 16-bit stack pointer
- Six 8-bit registers arranged in pairs: BC, DE, HL
- Requires +5V supply to operate at 3.2 MHZ single phase clock

8085 microprocessor consists of the following functional units:

Accumulator (A):

It is an 8-bit register which is used to perform arithmetical and logical operation. It stores the output of any operation. It also works as registers for I/O accesses. It can be one of the operand in the instruction. It is connected to internal data bus & ALU.

Temporary register:

It is an 8-bit register, which holds the temporary data of arithmetic and logical operations. It is connected to internal data bus & ALU. There are two additional temporary registers W and Z included in register array which are used to hold 8-bit data during the execution of some instructions. All are not available to the programmer as they are used internally.

General purpose registers:

There are 6 general purpose 8-bit registers in 8085 processor, i.e. B, C, D, E, H & L included in register array. Generally, theses registers are not used for storing the data permanently. They carry the 8-bits data and are used only during the execution of the instructions. These registers can also be used to carry the 16 bits data by making the pair of 2 registers. The valid register pairs available are BC, DE HL. All are available to the programmer.

Flag Register:

It is an 8-bit register included in register array. It consists of 5 flip-flops which changes its status according to the result stored in an accumulator. It is also known as status registers. It is connected to the ALU. There are five flip-flops in the flag register are as follows:

The bit position of the flip flop in flag register is:

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z		AC		P		CY

All of the five flip-flops are set and reset according to the stored result in the accumulator.

1. Sign flag (S):

If D7 of the result is 1 then sign flag is set otherwise reset. As we know that a bit on the D7 always decides the sign of the number.

if D7 is 1: the number is negative.

if D7 is 0: the number is positive.

2. Zeros flag (Z):

If the result stored in an accumulator is zero then this flip flop is set as 1 otherwise it is reset and also if the result of any arithmetic or logical operations is zero its set as 1(result of these operation can be in any registers).

3. Auxiliary carry flag (AC):

If any carry goes from D3 to D4 in the output then it is set otherwise it is reset. This flag is used only internally for BCD operations and is not available for the programmer to change the sequence of a program with a jump instruction.

4. Parity flag (P):

If the number of 1's is even in the output stored in the accumulator then it is set otherwise it is reset for the odd number of 1's.

5. Carry flag(C):

If the result stored in an accumulator generates a carry or borrow in its final output then it is set otherwise it is reset. It is also used as a borrow flag for subtraction.

Program counter (PC):

It is a 16-bit register used to store the memory addresses. The size of this register is 16-bits because the memory addresses are 16-bits. The microprocessor uses the PC register to sequence the execution of the instructions. The function of the PC is to point to the memory address from which the next byte is to be fetched. Microprocessor increments the program counter whenever an instruction is being executed, so that the program counter points to the memory address of the next instruction that is going to be executed.

Stack pointer (SP):

It is also a 16-bit register used to store the memory addresses. it points to a memory address in R/W memory; called as stack Generally, stack is a reserved portion of memory where information can be stores or taken back together. It is always incremented / decremented by 2 during **push** and **pop** operations.

Instruction registers (IR):

It is an 8-bit register. When an instruction is fetched from memory then it is stored in this register. It is not programmable and cannot be accessed through any instruction.

Instruction Decoder:

Instruction decoder identifies the instructions. It takes the information from instruction register and decodes the instruction to be performed.

Arithmetic and logic unit:

The ALU performs the actual arithmetic and logical operation such as 'add', 'subtract', 'AND', 'OR', etc. on 8-bit data. Uses data from memory or register and from Accumulator to perform arithmetic or logical operations and always stores the result of operation in the Accumulator.

Timing and control unit:

It provides timing and control signals to the microprocessor to perform the various operation. It has three control signals. It controls all external and internal circuits. It operates with reference to clock signal. It synchronizes all the data transfers. There are three control signals:

1. ALE signal (Address Latch Enable):

This is positive going pulse generated every time the 8085 begin an operation (machine cycle): it means that the bits on AD0 – AD7 are address bits. This signal is primarily used to latch the low-order address from the multiplexed bus and generates a separate set of eight address lines A0 – A7.

2. RD signal (Read):

This is active low used for reading operation. This signal indicates that the selected I/O or memory device location to be read and data are available on data bus.

3. WR signal (Write):

This is an active low used for writing operations. This signal indicates that the s data available on data bus are to be written into selected I/O or memory location.

There are three status signals used in microprocessor S0, S1 and IO/M. It changes its status according the provided input to these pins.

Serial IO Control:

It controls the serial data communication by using these two pins: SID (Serial input data) and SOD (Serial output data).

Interrupt Control Unit:

It controls the interrupts during a process. When a microprocessor is executing a main program and whenever an interrupt occurs, the microprocessor shifts the control from the main program to process the incoming request. After the request is completed, the control goes back to the main program. There are 5 interrupt signals in 8085 microprocessor: INTR, RST 7.5, RST 6.5, RST 5.5, TRAP.

Address buffer and address-data buffer:

The content stored in the stack pointer and program counter is loaded into the address buffer and address-data buffer to communicate with the CPU. The memory and I/O chips are connected to these buses; the CPU can exchange the desired data with the memory and I/O chips.

Address bus and data bus:

Data bus carries the data to be stored. It is bidirectional, whereas address bus carries the address to where it should be stored, and it is unidirectional. It is used to transfer the data & address.

QUIZ:

1) What is Register? How many registers is in 8085 microprocessor?
2) What is Timing and Control Unit?
3) Give functionality of accumulator and program counter.

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____

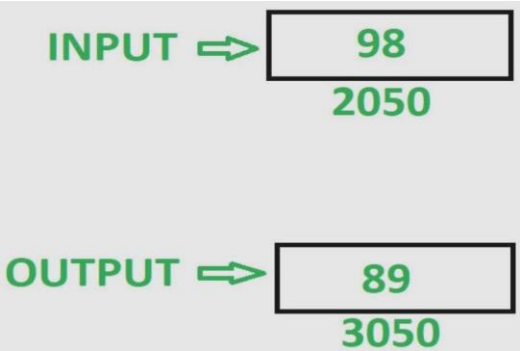
EXPERIMENT NO: 5 DATE: / /2026

TITLE: Write an assembly language code in GNUsim8085 to store number in reverse order in memory location.

OBJECTIVES: On completion of this experiment student will able to…

- know the concept of assembly language
- know the concept of how to write down the assembly program in GNUsim8085 for storing number in reverse order in memory location.

THEORY:



Assume that number to be reversed is stored at memory location 2050, and reversed number is stored at memory location 3050.

ALGORITHM:

- Load content of memory location 2050H in accumulator A.
- Use RLC instruction to shift the content of A by 1 bit without carry. Use this instruction 4 times to reverse the content of A.
- Store content of A in memory location 3050H.

PROGRAM:

MEMORY ADDRESS	MNEMONICS	COMMENT
2000H	**LDA 2050H**	A ← M [2050H]
2003H	**RLC**	Rotate content of accumulator left by 1 bit
2004H	**RLC**	Rotate content of accumulator left by 1 bit
2005H	**RLC**	Rotate content of accumulator left by 1 bit
2006H	**RLC**	Rotate content of accumulator left by 1 bit
2007H	**STA 3050H**	M [3050H] ← A
200AH	**HLT**	END

EXPLANATION: – Register A used:

LDA 2050H **:** load value of memory location 2050H in Accumulator A.

RLC **:** Rotate content of accumulator left by 1 bit

RLC **:** Rotate content of accumulator left by 1 bit

RLC **:** Rotate content of accumulator left by 1 bit

RLC **:** Rotate content of accumulator left by 1 bit

STA 3050H **:** store content of A in memory location 3050H.

HLT **:** stops executing the program and halts any further execution.

Another Alternative and simple way to execute the above program and the magic of 8085 rotate instructions:

This can be done by using **4 times RRC instruction.**

LDA 2050H

// 4 RRC do same work as 4 RLC. So, we can use anyone alternatively //

RRC

RRC

RRC

RRC

// after this 4th RRC instruction our 8-bit number is reversed and stored in accumulator **//**

STA 3050H

HLT

Step-by-Step and Bit-by-Bit explanation of the Above Program with an example:

Example:

98H in Binary Written as: 1001 1000

RLC 1st Time: 0011 0001 {Carry Flag = 1}

RLC 2nd Time: 0110 0010 {Carry Flag = 0}

RLC 3rd Time: 1100 0100 {Carry Flag = 0}

RLC 4th Time: 1000 1001 {Carry Flag = 1}

Converted Number after 4th RLC: 1000 1001 [89H]

Hence our number is reversed from 98H to 89H.

Example:

98H in Binary Written as: 1001 1000

RRC 1st Time: 0100 1100 {Carry Flag = 0}

RRC 2nd Time: 0010 0110 {Carry Flag = 0}

RRC 3rd Time: 0001 0011 {Carry Flag = 0}

RRC 4th Time: 1000 1001 {Carry Flag = 1}

Converted Number after 4th RRC: 1000 1001 [89H]

Hence our number is reversed from 98H to 89H.

Hence Instead of 4 RLC, we can also use 4 RRC instructions in our code alternatively.

EXERCISE:

1) Write an assembly language program for exchange the contents of memory locations 2050H and 2051H.
2) Write an assembly language program to swap the nibbles of the given number.

QUIZ:

1) What is RLC and ROR Instruction in 8085?
2) What is purpose of STA and HLT instruction?

EVALUATION:

Involvement (4)	Understanding / Problem solving (3)	Timely Completion (3)	Total (10)

Signature with date: _____

EXPERIMENT NO: 6 DATE: / /2026

TITLE: Write an assembly language code in GNU sim8085 to implement arithmetic instruction.

OBJECTIVES: On completion of this experiment student will able to…

- know the concept of assembly language
- know the concept of arithmetic instruction in GNUsim8085

THEORY:

Arithmetic Instructions are the instructions which perform basic arithmetic operations such as addition, subtraction and a few more. In 8085 microprocessor, the destination operand is generally the accumulator. Following is the table showing the list of arithmetic instructions:

Opcode	Operand	Explanation	Example
ADD	R	A = A + R	ADD B
ADD	M	A = A + MC	ADD 2050
ADI	8-bit data	A = A + 8-bit data	ADI 50
ADC	R	A = A + R + prev. carry	ADC B
ADC	M	A = A + Mc + prev. carry	ADC 2050
ACI	8-bit data	A = A + 8-bit data + prev. carry	ACI 50
SUB	R	A = A – R	SUB B
SUB	M	A = A – MC	SUB 2050
SUI	8-bit data	A = A – 8-bit data	SUI 50
SBB	R	A = A – R – prev. carry	SBB B
SBB	M	A = A – MC -prev. carry	SBB 2050
SBI	8-bit data	A = A – 8-bit data – prev. carry	SBI 50
INR	R	R = R + 1	INR B
INR	M	M = MC + 1	INR 2050
INX	RP	RP. = RP + 1	INX H
DCR	R	R = R – 1	DCR B
DCR	M	M = MC – 1	DCR 2050
DCX	RP	RP = RP – 1	DCX H
DAD	RP	HL = HL + RP	DAD H

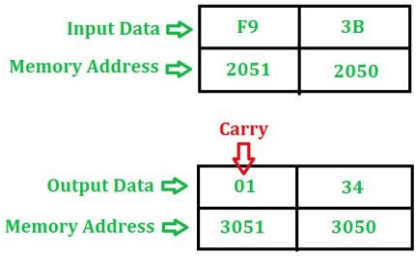
In the above table,

R stands for register **M** stands for memory

Mc stands for memory contents **RP** stands for register pair

EXAMPLE-1:

Write an Assembly Language Program to add two 8-bit numbers. Assume that two 8-bit numbers stored at memory locations 2050H and 2051H and the result is more than 8-bit which are stored in memory locations 3050H (LSB) and 3051H (MSB(Carry)).



ALGORITHM:

- Load the first number from memory location 2050H to accumulator.
- Move the content of accumulator to register H.
- Load the second number from memory location 2051H to accumulator.
- Then add the content of register H and accumulator using "ADD" instruction and storing result at 3050H
- The carry generated is recovered using "ADC" command and is stored at memory location 3051H.

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	**LDA 2050H**	A← [2050H]
2003H	**MOV H, A**	H ← A
2004H	**LDA 2051H**	A ← [2051H]
2007H	**ADD H**	A ← A+H
2008H	**MOV L, A**	L ← A
2009H	**MVI A 00**	A ← 00
200BH	**ADC A**	A ← A + A + carry
200CH	**MOV H, A**	H ← A
200DH	**SHLD 3050H**	[3050H]← L, [3051H]← H
2010H	**HLT**	

EXPLANATION:

LDA 2050H ; moves the contents of 2050H memory location to the accumulator.

MOV H, A ; copy the content of Accumulator to register H to A.

LDA 2051H ; moves the contents of 2051H memory location to the accumulator.

ADD H ; adds contents of A (Accumulator) and H register (F9). The result is stored in A itself. For all arithmetic instructions A is by default an operand and A stores the result as well.

MOV L, A ; copies contents of A (34) to L.

MVI A 00 ; moves immediate data (i.e., 00) to A.

ADC A ; adds contents of A (00), contents of register specified (A) and carry (1). As ADC is also an arithmetic operation; A is by default an operand and A stores the result as well.

MOV H, A ; copies contents of A (01) to H.

SHLD 0003 ; moves the contents of L register (34) in 3050H memory location and contents of H register (01) in 3051H memory location.

HLT ; stops executing the program and halts any further execution.

EXAMPLE-2:

Write an Assembly language program to add two 8-bit numbers 24H and 46H.

ALGORITHM:

- Move the first number 24H in register A.
- Move the first number 46H in register B.
- Then add the content of register B with A using "ADD B" instruction and result by default stored in register A.

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	**MVI A, 24H**	A← 24H
2002H	**MVI B, 46H**	B ← 46H
2004H	**ADD B**	A ← A+B
2005H	**HLT**	

EXPLANATION:

MVI A, 24H ; Copy data value 24H into register A

MVI B, 46H ; Copy data value 46H into register B

ADD B ; add the value of register A with B and result by default stored into A

HLT ; stops executing the program and halts any further execution.

EXAMPLE-3:

Write an Assembly language Program to add two 8-bit numbers 99H and 99H. Assume that result can be more than 8-bits.

ALGORITHM:

- Move the first number 98H in register A.
- Move the second number 99H in register E.
- Then add the content of register B with A using "ADD E" instruction and result by default stored in register A (LSB).
- After addition, if carry is generated then stored in register B (MSB) by incrementing it by 1 (B content before incrementing is 00H).

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	**MVI A, 98H**	A← 98H
2002H	**MVI E, 99H**	E ← 99H
2004H	**ADD B**	A ← A+B
2005H	**JNC L1**	If CY = 0, then go to L1
2008H	**MVI B, 00H**	B← 00H
200AH	**INR B**	B← 01H
200BH	**L1: HLT**	

```
        MVI A, 98H    ; Copy data value 98H into register A
        MVI E, 99H    ; Copy data value 99H into register E
        ADD E         ; Add the value of register A with E and result by default stored into A
        MOV C, A      ; Copy the result stored in register A into C
        JNC L1        ; Check the carry bit after addition: if CY = 1, it is stored into register B
        MVI B, 00H    ; Copy 00H in register B
        INR B         ; Increment content B by 1 for carry storage
L1:     HLT           ; Stops executing the program and halts any further execution.
```

EXAMPLE-4:

Write an Assembly language program to subtract two 8-bit numbers 46H and 24H.

ALGORITHM:

- Move the first number 46H in register A.
- Move the first number 24H in register B.
- Then subtract the content of register B from A using "SUB B" instruction and result by default stored in register A.

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	**MVI A, 46H**	A← 46H
2002H	**MVI B, 24H**	B ← 24H
2004H	**SUB B**	A ← A - B
2005H	**HLT**	

EXPLANATION:

MVI A, 46H ; Copy data value 46H into register A

MVI B, 24H ; Copy data value 24H into register B

SUB B ; Subtract the value of register B from B and result by default stored into A

HLT ; stops executing the program and halts any further execution.

EXERCISE:

1) Write an assembly program for subtract two number with use of previous carry.

2) Write an assembly program to take a number from memory location and increment the value two times and store the result in memory location.

QUIZ:

1) What is Usefulness of ADI and ACI instruction?

2) Explain DAD and DCX instruction?

EVALUATION:

Involvement (4)	Understanding / Problem solving (3)	Timely Completion (3)	Total (10)

Signature with date: _____

EXPERIMENT NO: 7 DATE: / /2026

TITLE: Write an assembly language code in GNUsim8085 to find the factorial of a number.

OBJECTIVES: On completion of this experiment student will able to…

- know the concept of assembly language
- know the concept of how to write down the assembly program in GNUsim8085 for the factorial of a number.

THEORY:

EXAMPLE-1:

Write an assembly language program to multiply two 8-bit numbers stored at address 2050H and 2051H. Result is stored at address 3050H and 3051H. Starting address of program is taken as 2000.

Input Data ⇨	07	43
Memory Address ⇨	2051	2050

Output Data ⇨	01	D5
Memory Address ⇨	3051	3050

ALGORITHM:

- We are taking adding the number 43 seven (7) times in this example.
- As the multiplication of two 8-bit numbers can be maximum of 16 bits so we need register pair to store the result.

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	**LHLD 2050H**	A← [2050H], L← [2050H]
2003H	**XCHG**	H ↔ D, L ↔ E
2004H	**MOV C, D**	C ← D
2005H	**MVI D 00H**	D ← 00H
2007H	**LXI H 0000H**	H←00H, L←00H
200AH	**L1: DAD D**	HL← HL + DE
200BH	**DCR C**	C ← C - 1
200CH	**JNZ L1**	If Zero Flag ≠ 0, go to Label L1
200FH	**SHLD 3050H**	H→3051H, L→3050H
2012H	**HLT**	

EXPLANATION:

Registers used: A, H, L, C, D, E

```
        LHLD 2050H    ; loads content of 2051H in H and content of 2050H in L
        XCHG          ; exchanges contents of H with D and contents of L with E
        MOV C, D      ; copies content of D in C
        MVI D 00H     ; assigns 00H to D
        LXI H, 0000H  ; assigns 00H to H and 00 to L
L1:     DAD D         ; adds HL and DE and assigns the result to HL
        DCR C         ; decrements C by 1
        JNZ L1        ; jumps label L1 if zero flag ≠ 0
        SHLD 3050H    ; stores value of H at memory location 3051H and L at 3050H
        HLT           ; stops executing the program and halts any further execution
```

Another approach:

We can do multiplication of two 8-bit numbers without using DAD and XCHG command.

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	LXI H, 2050H	2050H ← HL
2003H	MOV B, M	B ← M
2004H	INX H	HL ← HL + 1
2005H	MOV C, M	C ← M
2006H	MVI A, 00H	A ← 00H
2008H	TOP: ADD B	A ← A + B
2009H	DCR C	C ← C - 1
200AH	JNZ TOP	If Zero Flag ≠ 0, go to Label TOP
200DH	INX H	HL ← HL + 1
200EH	MOV M, A	M ← A
200FH	HLT	

EXPLANATION:

Registers A, H, L, C, B are used for general purpose.

```
        LXI H, 2050  ; load the HL pair register with the address 2050 of memory location.
        MOV B, M     ; copies the content of memory into register B.
```

```
        INX H       ; increment the address of HL pair by one and make it 2051H.
        MOV C, M    ; copies the content of memory into register C.
        MVI A,00H   ; assign 00 to A.
TOP:    ADD B       ; add the content of A with B and by default store the result in A.
        DCR C       ; decrement the register C.
        JNZ TOP     ; jumps on top till C doesn't becomes 0.
        INX H       ; increment the address of HL pair by one and make it 2052H.
        MOV M, A    ; copies the content of A which is our answer to register M.
        HLT         ; stops executing the program and halts any further execution.
```

EXAMPLE-2:

Write an assembly Language Program to find out factorial of the 8-bit number storer in memory location 2050H and result stored at memory location 3050H.

THEORY:

The factorial of 5 would be 78H. The logic is **5! = 5 X 4 X 3 X 2 X 1 = 120 = 78H**

So, add 5 four times, the result (i.e. 20) add three times and so on.....(5+5+5+5), (20+20+20), (60+60) and convert 120 to hex is 78H

Without Subroutine:

PROGRAM:

Memory Address	Mnemonics	Comment
2000H	LXI H, 2050H	HL ← 2050H
2003H	MOV B, M	B ← M
2004H	MOV C, M	C ← M
2005H	DCR B	B ← B - 1
2006H	L2: MOV D, B	D ← B
2007H	MVI A, 00H	A ← A + B
2009H	L1: ADD C	A ← A + B
200AH	DCR D	D ← D – 1
200BH	JNZ L1	If Zero Flag ≠ 0, go to Label L1
200EH	MOV C, A	C ← A
200FH	DCR B	B ← B – 1
2010H	JNZ L2	If Zero Flag ≠ 0, go to Label L2
2013H	STA 3050H	[3050H] ← A
2016H	HLT	

EXPLANATION:

Registers A, B, C, D, H, L are used for general purpose.

```
        LXI H, 2050H    ; load the HL pair register with the 2050H memory location.
        MOV B, M        ; copies the content of memory into register B.
        MOV C, M        ; copies the content of memory into register C.
        DCR B           ; Decrement the register B.
        MOV D, B        ; copies the content of B into register D.
        MVI A,00H       ; assign 00 to A.
L1:     ADD C           ; add the content of A with C and by default store the result in A.
        DCR D           ; decrement the register D.
        JNZ L1          ; jumps on L1 till D doesn't becomes 0.
        MOV A, C        ; copies the content of C into register A.
        DCR B           ; Decrement the register B.
        JNZ L2          ; jumps on L1 till B doesn't becomes 0.
        STA 3050H       ; copies the content of A which is our answer to 3050H Location.
        HLT             ; stops executing the program and halts any further execution.
```

EXCERCISE:

1) Write an assembly program to square of a given number.

2) Write an assembly program for the 8-bit division.

QUIZ:

1) Give difference between MVI and MOV instruction.

2) What is Flag bit?

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____

EXPERIMENT NO: 8 DATE: / / 2025

TITLE: Write an assembly language code in GNUsim8085 to implement logical instructions.

OBJECTIVES: On completion of this experiment student will able to…

- know the concept of assembly language
- know the concept of logical instruction in GNUsim8085

THEORY:

Logical instructions in the 8085 microprocessor are a set of instructions that perform logical operations on data in registers and memory. Logical operations are operations that manipulate the bits of data without affecting their numerical value. These operations include AND, OR, XOR, and NOT. In the 8085 microprocessor, the destination operand is always the accumulator and logical operation works on a bitwise level. Following is the table showing the list of logical instructions:

OPCODE	OPERAND	DESTINATION	EXAMPLE
ANA	R	A = A AND R	ANA B
ANA	M	A = A AND M	ANA 2050H
ANI	8-bit data	A = A AND 8-bit data	ANI 50H
ORA	R	A = A OR R	ORA B
ORA	M	A = A OR M	ORA 2050H
ORI	8-bit data	A = A OR 8-bit data	ORI 50H
XRA	R	A = A XOR R	XRA B
XRA	M	A = A XOR M	XRA 2050H
XRI	8-bit data	A = A XOR 8-bit data	XRI 50H
CMA	None	A = 1's compliment of A	CMA
CMP	R	Compares R with A and triggers the flag register	CMP B
CMP	M	Compares Mc with A and triggers the flag register	CMP 2050H
CPI	8-bit data	Compares 8-bit data with A and triggers the flag register	CPI 50H
RRC	none	Rotate accumulator right without carry	RRC
RLC	none	Rotate accumulator left without carry	RLC
RAR	none	Rotate accumulator right with carry	RAR
RAL	none	Rotate accumulator left with carry	RAR
CMC	none	Compliments the carry flag	CMC
STC	none	Sets the carry flag	STC

In the table,

R stands for register **M** stands for memory

EXAMPLE-1:

Write an Assembly Language Program to perform all logical operation on 45H and AAH data. Store the results in memory location starts from 2050H onwards.

```
MVI B, 55H    ; move the data 55H to register B.
MVI C, 0AAH   ; move the data 55H to register C.
MOV A, B      ; copy the content of B to A.
CMA           ; complement (NOT) on the data in A and by default stored in A.
STA 2050H     ; copy the content of A which is NOT answer to 2050H Location.
MOV A, B      ; copy the content of B to A.
ANA C         ; AND operation on contents of A and B and by default stored in A.
STA 2051H     ; copy the content of A which is AND answer to 2051H Location.
CMA           ; NOT operation on AND result to obtain NAND operation result in A.
STA 2052H     ; copy the content of A which is NAND answer to 2052H Location.
MOV A, B      ; copy the content of B to A.
ORA C         ; OR operation on contents of A and B and by default stored in A.
STA 2053H     ; copy the content of A which is OR answer to 2053H Location.
CMA           ; NOT operation on OR result to obtain NOR operation result in A.
STA 2054H     ; copy the content of A which is NOR answer to 2054H Location.
MOV A, B      ; copy the content of B to A.
XRA C         ; EXOR operation on contents of A and B and by default stored in A.
STA 2055H     ; copy the content of A which is EXOR answer to 2055H Location.
CMA           ; NOT operation on EX-OR result to obtain EXNOR operation result in A.
STA 2056H     ; copy the content of A which is EXNOR answer to 2056H Location.
HLT           ; stops executing the program and halts any further execution.
```

EXAMPLE-2:

Write an Assembly Language Program to perform all basic logical operations (NOT, AND, OR and EX-OR).

```
LXI H, 2050H  ; load the HL pair register with the 2050H memory location.
MOV B, M      ; copies the content of memory into register B.
INX H         ; increment the address of HL pair by one and make it 2051H.
```

```
MOV C, M    ; copies the content of memory into register C.
MOV A, B    ; copies the content of B into register A.
CMA         ; complement (NOT) on the data in A and by default stored in A.
STA 3050H   ; copies the content of A which is NOT answer to 3050H Location.
MOV A, B    ; copies the content of B into register A.
ANA C       ; AND operation on contents of A and B and by default stored in A.
STA 3051H   ; copies the content of A which is AND answer to 3051H Location.
MOV A, B    ; copies the content of B into register A.
ORA C       ; OR operation on contents of A and B and by default stored in A.
STA 3052H   ; copies the content of A which is OR answer to 3052H Location.
MOV A, B    ; copies the content of B into register A.
XRA C       ; EX-OR operation on contents of A and B and by default stored in A.
STA 3053H   ; copies the content of A which is EX-OR answer to 3050H Location.
HLT         ; stops executing the program and halts any further execution.
```

EXAMPLE-3:

Write an Assembly Language Program to count odd numbers in an array of ten 8-bit numbers stored in memory location starting from 2050H onwards.

```
    LXI H, 2050H  ; load the HL pair register with the 2050H memory location.
    MVI B, 00H    ; Initialized B with 00H (for storing odd number count)
    MVI C, 0AH    ; Initialized C with 0AH (as a counter)
L2: MOV A, M      ; copy the content of memory into A
    ANI 01H       ; perform AND operation of A with 01H and store the result in A.
    JZ L1         ; jump if ZF = 1 to label L1 (if number is even).
    INR B         ; increment B by 01 (if number is odd) (for storing odd number count).
    INX H         ; increment indirect memory location M by 01H (for next array data)
L1: DCR C         ; decrement C by 01 (as a counter for counting array data).
    JNZ L2        ; jump if ZF = 0 to label L2 (if counter of array data is not 0).
    HLT           ; stops executing the program and halts any further execution.
```

EXAMPLE-3:

Write an Assembly Language Program to find out largest number from an array of six 8-bit numbers stored in memory location starting from 3050H onwards.

```
    LXI H, 3050H  ; load the HL pair register with the 3050H memory location.
```

```
    MVI C, 05H      ; Initialized C with 05H (as a counter)
    MOV A, M        ; copy the content of memory into A
L2: INX H           ; increment indirect memory location M by 01H (for next array data)
    CMP M       ; compare A with M (nondestructive subtraction) and change status of flags.
    JNC L1          ; jump if CF = 0 to label L1 (if A > M).
    MOV A, M        ; copy the content of memory into A (as A < M)
L1: DCR C           ; decrement C by 01 (as a counter for counting array data).
    JNZ L2          ; jump if ZF = 0 to label L2 (if counter of array data is not 0).
    STA 2050H       ; copy the content of A which is largest answer to 2050H Location.
    HLT             ; stops executing the program and halts any further execution.
```

EXERCISE:

1) Write an Assembly Language Program to count even numbers in an array of nine 8-bit numbers stored in memory location starting from 3050H onwards.
2) Write an Assembly Language Program to find out smallest number from an array of six 8-bit numbers stored in memory location starting from 2050H onwards.

QUIZ:

1) What is Usefulness of JNZ, DCR and CMP instructions?
2) What is full form of the INR, INX and JZ instructions?

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____

EXPERIMENT NO: 9 DATE: / /2026

TITLE: Design ALU using Logisim.

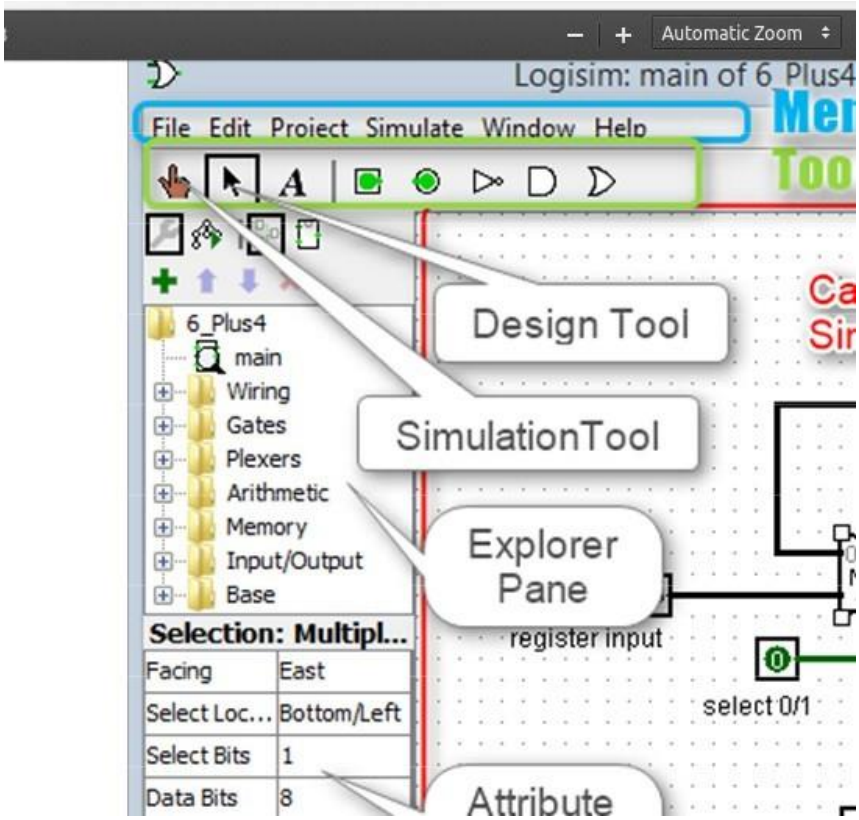
OBJECTIVES: On completion of this experiment student will able to…

- know the concept of Logisim

THEORY:

Logisim Overview:

Logisim is an educational and user-friendly tool. It mainly consists of an interactive graphical schematic editor and logic simulator. Logisim has a layout similar to most available software tools (Figure 1).



The main window consists of the following items:

Toolbar: contains short cuts to several commonly used items

The simulation tool: shaped like a hand, is used in simulation mode to alter input pins.

The design tool: It is used while designing the circuit.

The input pin: green circle surrounded by a square box, is used to send a signal through a wire. When placing the input on the canvas it initializes the input to logic 1 or 0. The number of bits can be increased in the Attribute Table.

The output pin: green circle surrounded by a circular shape, is used to observe the output from a gate or a block. The output pin toggles in real time as long as the simulation is enabled from the menu bar: Simulate > Simulation Enabled

Explorer Pane: This pane contains the list of wiring, gates, multiplexers and other components that are available for digital design in Logisim.

Attribute Table: Gives detailed attributes of digital design components (e.g., AND, OR, XOR gates). The attribute table allows you to alter the number of inputs/outputs that a digital component may have.

Canvas: The canvas is the area for you to create your digital circuits. In this area you may simulate your circuits while designing in real time.

Circuit Design:

Logisim operates in two modes: Design and Simulate. Logisim is first operated in edit mode while designing a logic circuit, then in simulate mode to see how the circuit would actually work. Edit Mode

Edit Mode:

To use the edit mode, select the Design tool.

Select then a component in the library on the left. To add it in your design, simply drag and drop the desired component in the canvas:

To insert a logic gate, expand the folder Gates and click on the desired logic gate. Once the gate is selected, the mouse will turn into the shape of the selected gate. Place the gate in the canvas or circuit area on the right.

Each selected component has modifiable attributes in the attribute area. For example, for an AND gate, the number of input signals, the number of bits per input or output and the size of the component can all be modified.

It is also possible to copy and paste of one or more components in the canvas. In this case, the components retain all the attributes previously defined.

To connect the gates, select the arrow icon on top. Then drag from the output of one gate to the input of another gate. You can connect to any of the small blue dots on the gate symbol.

To create an input to the circuit, select the “Add Pin" icon with an outline of a square. Similarly, add an output to the circuit by using the “Add Pin" icon with an outline of a circle. To assign a name to the input/output pin, click on the pin while the arrow icon is selected. You may then add the label for the pins.

While components are added to the circuit, notice how the color of the wire changes.

Table 10.1: Logisim Wire Color Co

Wire Color	Meaning
dark green	logical `0'
light green	logical `1'
blue	unknown valu
gray	unconnected w
black	wire has more

Full Adder Design

In Logisim, a digital circuit may be designed using either one of two approaches:

Drawing schematic: the user has to do the design by choosing the necessary components as per his design, based on the digital schematic.

Analyzing the circuit, by providing the truth tables with necessary inputs and outputs and let Logisim do the design.

Schematic Design: A set up the truth table for a simple Full Adder.

Let A B and Ci be the inputs, S and Co the outputs.

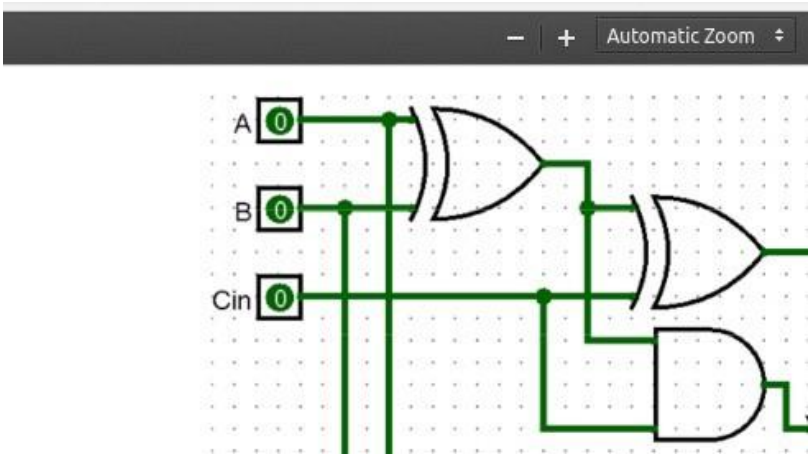
A	B	Ci
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0

Derive the logic equations for S and Cout:

$S = A \oplus B \oplus Ci$

$Cout = (A \wedge B) + (Ci \wedge (A \oplus B))$.

Finally, design the circuit based on the above equations.



The ALU (Arithmetic Logic Unit) Design:

Arithmetic Logic Unit (ALU), which is capable of performing logical operations (e.g. AND, OR, Ex-OR, Invert etc.) in addition to the arithmetic operations (e.g. Addition, Subtraction etc.). The control unit supplies the data required by the ALU from memory, or from input devices, and directs the ALU to perform a specific operation based on the instruction fetched from the memory. ALU is the "calculator" portion of the computer.

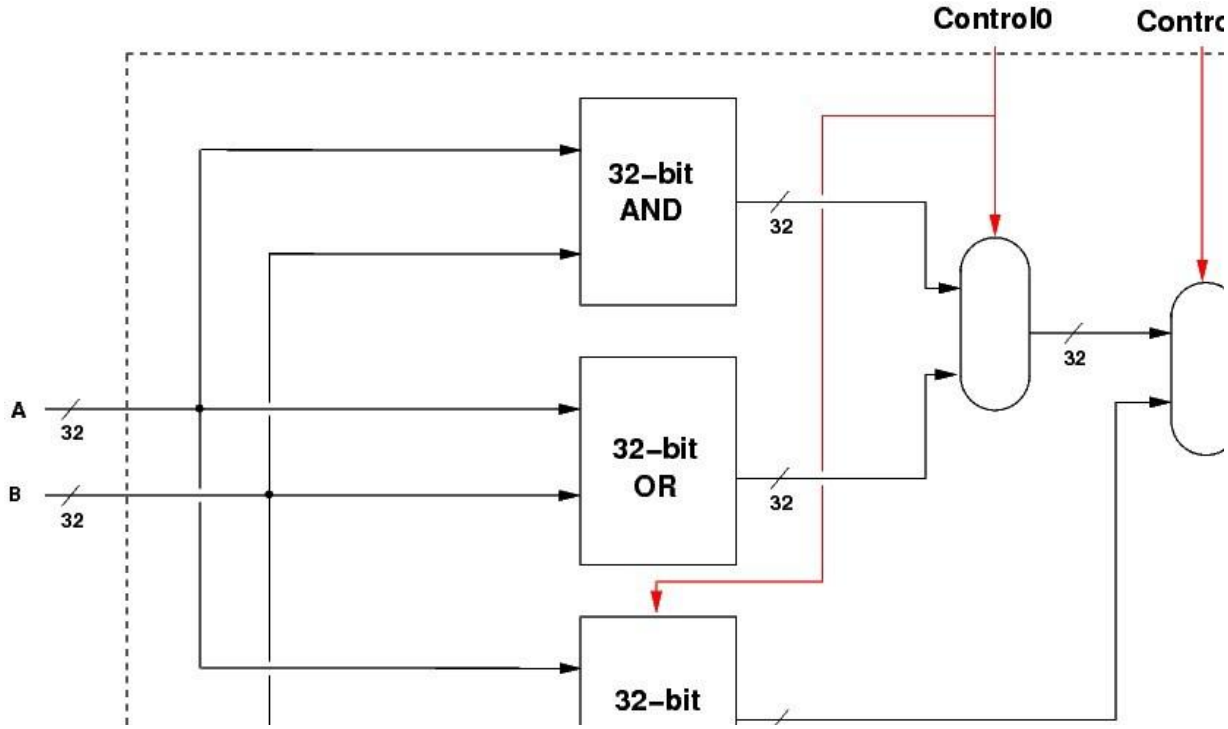
An Arithmetic Logic Unit (ALU) is a major component of the central processing unit of a computer system. It does all processes related to arithmetic and logic operations that need to be done on instruction words. In some microprocessor architectures, the ALU is divided into the arithmetic unit (AU) and the logic unit (LU).

An ALU can be designed for calculate many different operations. When the operations become more and more complex, then the ALU will also become more and more expensive and also takes up more space in the CPU and dissipates more heat. That is why engineers make the ALU powerful enough to ensure that the CPU is also powerful and fast, but not so complex as to become prohibitive in terms of cost and other disadvantages.

Different operation as carried out by ALU can be categorized as follows –

- **logical operations**: These include operations like AND, OR, NOT, XOR, NOR, NAND, etc.
- **Bit-Shifting Operations**: This pertains to shifting the positions of the bits by a certain number of places either towards the right or left, which is considered a multiplication or division operations.

- **Arithmetic operations:** This refers to bit addition and subtraction. Although multiplication and division are sometimes used, these operations are more expensive to make. Multiplication and subtraction can also be done by repetitive additions and subtractions respectively.



We now have three main units: a 32-bit bitwise AND unit, a 32-bit bitwise OR unit, and a 32-bit ADD/SUBTRACT unit with a control line, the logic to output carry, overflow, zero and negative. We can pass the inputs A and B to all three units, and perform all three operations in parallel.

The dotted line shows the interface to the ALU. Here we are use four logic gates: AND, OR, NOT and XOR. We are going to use another component, the multiplexor. The job of the multiplexor is to choose one of several inputs, based on a control line, and send the chosen input to the output.

This multiplexor above is a 1-bit wide 2-way multiplexor: 2 inputs, 1 bit wide each. If you add extra control lines, you can choose more inputs: 2 control lines is used in a 4-way multiplexor, 3 control lines in an 8-way multiplexor etc. and by using multiple multiplexors in parallel, you can make N-bit wide multiplexors.

QUIZ:

1) What is Design tool in Logisim simulator?

2) What is Attribute Table in Logisim simulator?

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____

EXPERIMENT NO: 10 DATE: / /2026

TITLE: Implement 16-bit single-cycle MIPS processor in Verilog HDL

OBJECTIVES: On completion of this experiment student will able to…

- know the concept of MIPS (Microprocessor without Interlocked Pipelined Stages) processor.

THEORY:

In this project, a 16-bit single-cycle MIPS processor is implemented in Verilog HDL. MIPS is an RISC processor, which is widely used by many universities in academic courses related to computer organization and architecture.

The Instruction Format and Instruction Set Architecture for the 16-bit single-cycle MIPS are as follows:

Name	Fields					Comments
Field Size	3-bits	3-bits	3-bits	3-bits	4-bits	all MIPS – L instructions 16-bit
R-format	op	rs	rt	rd	funct	Arithmetic instruction format
I-format	op	rs	rt	Address/immediate		Transfer, branch, immediate format
J-format	op	target address				Jump instruction format

Name	Format	Example					Comments
		3-bits	3-bits	3-bits	3-bits	4-bits	
Add	R	0	2	3	1	0	Add S1, S2, S3
Sub	R	0	2	3	1	1	Sub S1, S2, S3
And	R	0	2	3	1	2	And S1, S2, S3
Or	R	0	2	3	1	3	Or S1, S2, S3
Slt	R	0	2	3	1	4	Slt S1, S2, S3
Jr	R	0	7	0	0	8	Jr S7
Iw	I	4	2	1	7		Iw S1, 7 (S2)
Sw	I	5	2	1	7		Sw S1, 7 (S2)
Beq	I	6	1	2	7		Beq S1, S2, 7
Addi	I	7	2	1	7		Addi S1, S2, 7
J	J	2	500				J 1000
Jal	J	3	500				Jal 1000
Slti	I	1	2	1	7		Slti S1, S2, 7

1. Add: R[rd] = R[rs] + R[rt]
2. Subtract: R[rd] = R[rs] - R[rt]
3. And: R[rd] = R[rs] & R[rt]
4. Or: R[rd] = R[rs] | R[rt]
5. Slt: R[rd] = 1 if R[rs] < R[rt] else R[rd] = 0
6. Jr: PC= R[rs]
7. Lw: R[rt] = M[R[rs]+Sign Ext Imm]
8. Sw: M[R[rs]+Sign Ext Imm] = R[rt]
9. Beq: if(R[rs] = = R[rt]) PC=PC+1+BranchAddr
10. Addi: R[rt] = R[rs] + Sign Ext Imm
11. J: PC = Jump Addr
12. Jal: R [7] = PC+2; PC=Jump Addr
13. Slti: R[rt] = 1 if R[rs] < imm else 0

Sign Ext Imm = {9{immediate [6]}, imm

Jump Addr ={(PC+1[15:13], address}

Branch Addr = {7{immediate [6]}, immediate, 1'b0}

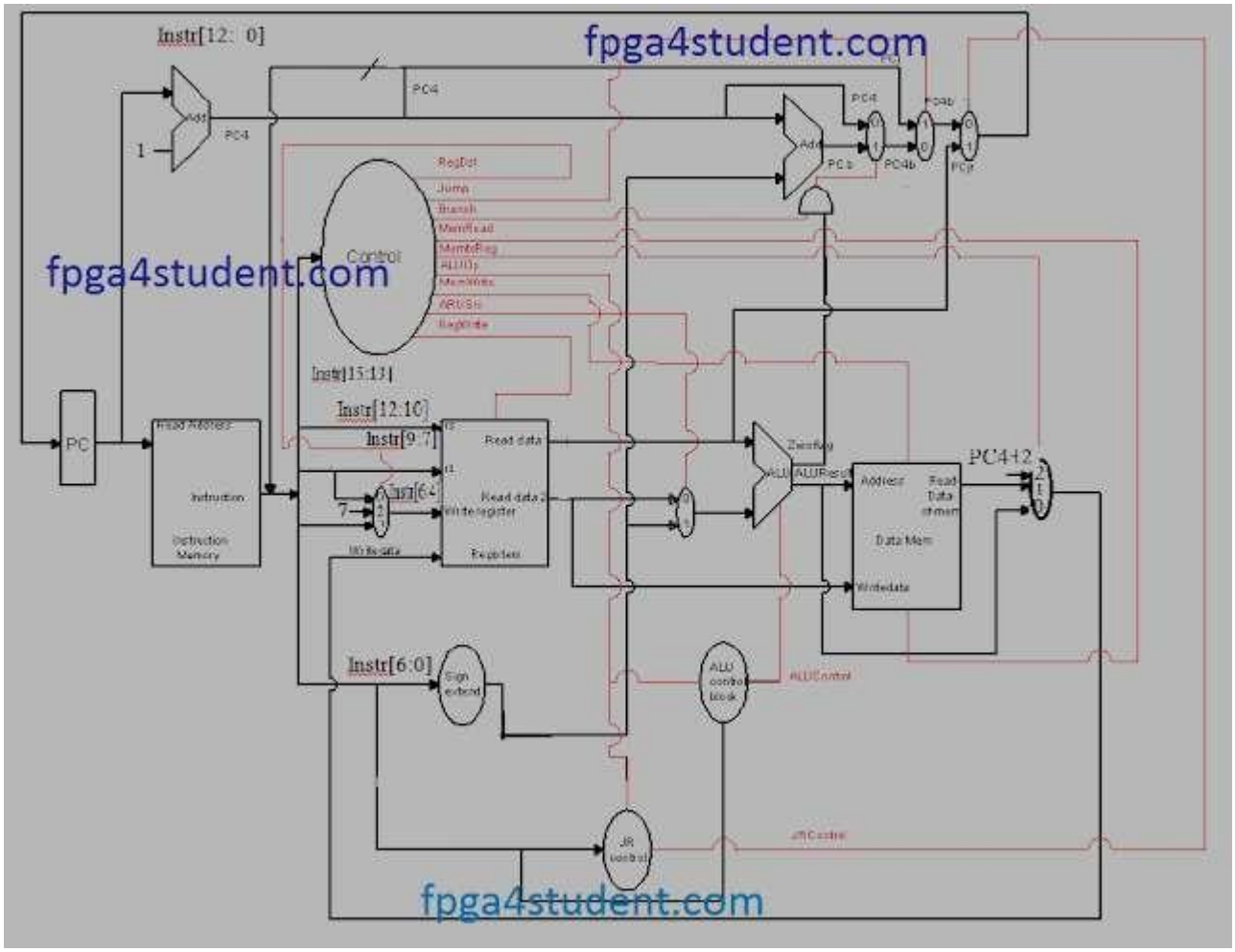
Based on the provided instruction set, the data-path and control unit are designed and implemented.

Control unit design:

Control signals									
Instruction	**Reg Dst**	**ALU Src**	**Mem to Reg**	**Reg Write**	**Mem Read**	**Mem Write**	**Branch**	**ALU Op**	**Jump**
R-type	1	0	0	1	0	0	0	00	0
Lw	0	1	1	1	1	0	0	11	0
Sw	0	1	0	0	0	1	0	11	0
Addi	0	1	0	1	0	0	0	11	0
Beq	0	0	0	0	0	0	1	01	0
J	0	0	0	0	0	0	0	00	1
Jal	2	0	2	1	0	0	0	00	1
Slti	0	1	0	1	0	0	0	10	0

ALU Control				
ALU op	**Function**	**ALU cnt**	**ALU Operation**	**Instruction**
11	xxxx	000	Add	Addi, Lw, Sw

01	xxxx	001	Sub	Beq
00	00	000	Add	R-type: Add
00	01	001	Sub	R-type: Sub
00	02	010	And	R-type: And
00	03	011	Or	R-type: OR
00	04	100	Slt	R-type: Slt
10	xxxxxx	100	Slt	I-type: Slti



After completing the design for the MIPS processor, it is easy to write Verilog code for the MIPS processor. The Verilog code for the whole design of the MIPS processor as follows:

1) Verilog code for ALU unit

2) Verilog code for register file

3) Verilog code for instruction memory

Verilog code for data memory:

```
// FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Submodule: Data memory in Verilog
 module data_memory
 (
    input      clk,
    // address input, shared by read and write port
    input      [15:0]    mem_access_addr,
    // write port
    input      [15:0]    mem_write_data,
    input                mem_write_en,
    input                mem_read,
    // read port
    output     [15:0]    mem_read_data
 );
    integer i;
    reg [15:0]     ram [255:0];
    wire [7:0]     ram_addr = mem_access_addr [8:1];
    initial begin
            for (i=0; i< 256; i=i+1)
                    ram[i] <= 16'd0;
    end
    always @ (posedge clk) begin
            if (mem_write_en)
                ram[ram_addr] <= mem_write_data;
    end
    assign mem_read_data = (mem_read==1'b1)? ram[ram_addr]: 16'd0;
 endmodule
```

Verilog code for ALU Control unit:

```
// FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Submodule: ALU Control Unit in Verilog
 module ALUControl (ALU_Control, ALUOp, Function);
 output reg [2:0] ALU_Control;
```

```
 input [1:0] ALUOp;
 input [3:0] Function;
 wire [5:0] ALUControlIn;
 assign ALUControlIn = {ALUOp, Function};
 always @(ALUControlIn)
casex (ALUControlIn)
      6'b11xxxx: ALU_Control=3'b000;
      6'b10xxxx: ALU_Control=3'b100;
      6'b01xxxx: ALU_Control=3'b001;
      6'b000000: ALU_Control=3'b000;
      6'b000001: ALU_Control=3'b001;
      6'b000010: ALU_Control=3'b010;
      6'b000011: ALU_Control=3'b011;
      6'b000100: ALU_Control=3'b100;
      default: ALU_Control=3'b000;
endcase
endmodule
// Verilog code for JR control unit
module JR_Control
  (   input [1:0]   alu_op,
      input [3:0]   funct,
      output        JRControl
  );
assign JRControl = ({alu_op, funct} = = 6'b001000)? 1'b1:1'b0;
endmodule
```

Verilog code for control unit:

```
// FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Submodule: Control Unit in Verilog
 module control
  (   input [2:0]             opcode,
      input                   reset,
      output reg [1:0]  reg_dst, mem_to_reg, alu_op,
      output                  reg jump, branch, mem_read, mem_write, alu_src, reg_write,
                              sign_or_zero
```

```
 );
always @ (*)
begin
    if (reset = = 1'b1) begin
            reg_dst = 2'b00;
            mem_to_reg = 2'b00;
            alu_op = 2'b00;
            jump = 1'b0;
            branch = 1'b0;
            mem_read = 1'b0;
            mem_write = 1'b0;
            alu_src = 1'b0;
            reg_write = 1'b0;
            sign_or_zero = 1'b1;
    end
    else begin
    case (opcode)
    3'b000:  begin                    // Add
                reg_dst = 2'b01;
                mem_to_reg = 2'b00;
                alu_op = 2'b00;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
                alu_src = 1'b0;
                reg_write = 1'b1;
                sign_or_zero = 1'b1;
                end
    3'b001:  begin                    // Sli
                reg_dst = 2'b00;
                mem_to_reg = 2'b00;
                alu_op = 2'b10;
                jump = 1'b0;
                branch = 1'b0;
                mem_read = 1'b0;
                mem_write = 1'b0;
```

```
            alu_src = 1'b1;
            reg_write = 1'b1;
            sign_or_zero = 1'b0;
            end
3'b010:  begin                      // j
            reg_dst = 2'b00;
            mem_to_reg = 2'b00;
            alu_op = 2'b00;
            jump = 1'b1;
            branch = 1'b0;
            mem_read = 1'b0;
            mem_write = 1'b0;
            alu_src = 1'b0;
            reg_write = 1'b0;
            sign_or_zero = 1'b1;
            end
3'b011:  begin                      // Jal
            reg_dst = 2'b10;
            mem_to_reg = 2'b10;
            alu_op = 2'b00;
            jump = 1'b1;
            branch = 1'b0;
            mem_read = 1'b0;
            mem_write = 1'b0;
            alu_src = 1'b0;
            reg_write = 1'b1;
            sign_or_zero = 1'b1;
            end
3'b100:  begin                      // Lw
            reg_dst = 2'b00;
            mem_to_reg = 2'b01;
            alu_op = 2'b11;
            jump = 1'b0;
            branch = 1'b0;
            mem_read = 1'b1;
            mem_write = 1'b0;
            alu_src = 1'b1;
```

```
            reg_write = 1'b1;
            sign_or_zero = 1'b1;
            end
    3'b101: begin                   // Sw
            reg_dst = 2'b00;
            mem_to_reg = 2'b00;
            alu_op = 2'b11;
            jump = 1'b0;
            branch = 1'b0;
            mem_read = 1'b0;
            mem_write = 1'b1;
            alu_src = 1'b1;
            reg_write = 1'b0;
            sign_or_zero = 1'b1;
            end
    3'b110: begin                   // Beq
            reg_dst = 2'b00;
            mem_to_reg = 2'b00;
            alu_op = 2'b01;
            jump = 1'b0;
            branch = 1'b1;
            mem_read = 1'b0;
            mem_write = 1'b0;
            alu_src = 1'b0;
            reg_write = 1'b0;
            sign_or_zero = 1'b1;
            end
    3'b111: begin                   // Addi
            reg_dst = 2'b00;
            mem_to_reg = 2'b00;
            alu_op = 2'b11;
            jump = 1'b0;
            branch = 1'b0;
            mem_read = 1'b0;
            mem_write = 1'b0;
            alu_src = 1'b1;
            reg_write = 1'b1;
```

```
            sign_or_zero = 1'b1;
            end
    default: begin
            reg_dst = 2'b01;
            mem_to_reg = 2'b00;
            alu_op = 2'b00;
            jump = 1'b0;
            branch = 1'b0;
            mem_read = 1'b0;
            mem_write = 1'b0;
            alu_src = 1'b0;
            reg_write = 1'b1;
            sign_or_zero = 1'b1;
            end
    endcase
    end
end
endmodule
```

Verilog code for the single-cycle MIPS processor:

```
// FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
 // Verilog code for 16-bit single cycle MIPS CPU
 module mips_16
  (input clk, reset,
  output [15:0] pc_out, alu_result        //reg3, reg4
  );
 reg [15:0] pc_current;
 wire signed [15:0] pc_next, pc2;
 wire [15:0] instr;
 wire [1:0] reg_dst, mem_to_reg, alu_op;
 wire jump, branch, mem_read, mem_write, alu_src, reg_write;
 wire [2:0] reg_write_dest;
 wire [15:0] reg_write_data;
 wire [2:0] reg_read_addr_1;
 wire [15:0] reg_read_data_1;
 wire [2:0] reg_read_addr_2;
```

```
wire [15:0] reg_read_data_2;
wire [15:0] sign_ext_im, read_data2, zero_ext_im, imm_ext;
wire JRControl;
wire [2:0] ALU_Control;
wire [15:0] ALU_out;
wire zero_flag;
wire signed [15:0] im_shift_1, PC_j, PC_beq, PC_4beq, PC_4beqj, PC_jr;
wire beq_control;
wire [14:0] jump_shift_1;
wire [15:0] mem_read_data;
wire [15:0] no_sign_ext;
wire sign_or_zero;
// PC
always @ (posedge clk or posedge reset)
begin
    if(reset)
        pc_current <= 16'd0;
    else
        pc_current <= pc_next;
end
// PC + 2
assign pc2 = pc_current + 16'd2;
// instruction memory
instr_mem instrucion_memory (.pc(pc_current), .instruction(instr));
// jump shift left 1
assign jump_shift_1 = {instr [13:0],1'b0};
// control unit
control control_unit (.reset (reset), .opcode(instr[15:13]),.reg_dst(reg_dst),
.mem_to_reg(mem_to_reg), .alu_op(alu_op),.jump(jump), .branch(branch),
.mem_read(mem_read), .mem_write(mem_write), .alu_src(alu_src), .reg_write(reg_write),
.sign_or_zero (sign_or_zero));
// multiplexer regdest
assign reg_write_dest = (reg_dst==2'b10)? 3'b111: ((reg_dst==2'b01)? instr [6:4]: instr
[9:7]);
// register file
assign reg_read_addr_1 = instr [12:10];
assign reg_read_addr_2 = instr [9:7];
```

```
register_file reg_file (.clk(clk), .rst (reset), .reg_write_en (reg_write),
.reg_write_dest(reg_write_dest),
.reg_write_data(reg_write_data),
.reg_read_addr_1(reg_read_addr_1),
.reg_read_data_1(reg_read_data_1),
.reg_read_addr_2(reg_read_addr_2),
.reg_read_data_2(reg_read_data_2));
//.reg3(reg3),
//.reg4(reg4));
// sign extend
assign sign_ext_im = {{9{instr [6]}}, instr [6:0]};
assign zero_ext_im = {{9{1'b0}}, instr [6:0]};
assign imm_ext = (sign_or_zero==1'b1)? sign_ext_im: zero_ext_im;
// JR control
JR_Control JRControl_unit ( .alu_op(alu_op), .funct (instr [3:0]), .JRControl(JRControl));
// ALU control unit
ALUControl ALU_Control_unit ( .ALUOp (alu_op), .Function(instr [3:0]), .ALU_Control
(ALU_Control));
// multiplexer alu_src
assign read_data2 = (alu_src==1'b1)? imm_ext: reg_read_data_2;
// ALU
alu alu_unit ( .a(reg_read_data_1), .b(read_data2), .alu_control (ALU_Control), .result
(ALU_out), .zero(zero_flag));
// immediate shift 1
assign im_shift_1 = {imm_ext [14:0],1'b0};
//
assign no_sign_ext = ~(im_shift_1) + 1'b1;
// PC beq add
assign PC_beq = (im_shift_1[15] == 1'b1)? (pc2 - no_sign_ext): (pc2 +im_shift_1);
// beq control
assign beq_control = branch & zero_flag;
// PC_beq
assign PC_4beq = (beq_control==1'b1)? PC_beq: pc2;
// PC_j
assign PC_j = {pc2[15], jump_shift_1};
// PC_4beqj
assign PC_4beqj = (jump == 1'b1)? PC_j: PC_4beq;
```

```
// PC_jr
assign PC_jr = reg_read_data_1;
// PC_next
assign pc_next = (JRControl==1'b1)? PC_jr: PC_4beqj;
// data memory
data_memory datamem ( .clk(clk), .mem_access_addr (ALU_out),
.mem_write_data (reg_read_data_2), .mem_write_en (mem_write), .mem_read(mem_read),
. mem_read_data(mem_read_data));
// write back
assign reg_write_data = (mem_to_reg == 2'b10)? pc2: ((mem_to_reg == 2'b01)?
mem_read_data: ALU_out);
// output
assign pc_out = pc_current;
assign alu_result = ALU_out;
endmodule
```

Verilog testbench code for the single-cycle MIPS processor:

```
`timescale 1ns / 1ps
// FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 16-bit MIPS Processor
// Testbench Verilog code for 16-bit single cycle MIPS CPU
 module tb_mips16;
// Inputs
reg clk;
reg reset;
// Outputs
wire [15:0] pc_out;
wire [15:0] alu_result;//, reg3, reg4;
// Instantiate the Unit Under Test (UUT)
mips_16 uut
(     .clk (clk),
      .reset (reset),
      .pc_out (pc_out),
      .alu_result (alu_result)
      //.reg3 (reg3),
      // .reg4(reg4)
);
```

```
initial begin
      clk = 0;
      forever #10 clk = ~clk;
end
initial begin
// Initialize Inputs
//$monitor ("register 3=%d, register 4=%d", reg3, reg4);
      reset = 1;
// Wait 100 ns for global reset to finish
      #100;
      reset = 0;
// Add stimulus here
end
endmodule
```

It is quite simple to verify the Verilog code for the single-cycle MIPS CPU by doing several simulations on **ModelSim or Xilinx ISIM** in order to see how the MIPS processor works. To fully verify the MIPS processor, it is needed to modify the instruction memory to simulate all the instructions in the instruction set architecture, and then check simulation waveform and memory to see if the processor works correctly as designed.

QUIZ:

1) What is 16-bit single cycle?
2) What is full form of FPGA?

EVALUATION:

Involvement (4)	**Understanding / Problem solving (3)**	**Timely Completion (3)**	**Total (10)**

Signature with date: _____